# APPLICATION STUDY I - DNN

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated
Intelligent
Systems Lab.

# CONTENT

- DNN components
- Parallelizing DNNs
- An extra (but a common) technique: layer fusion

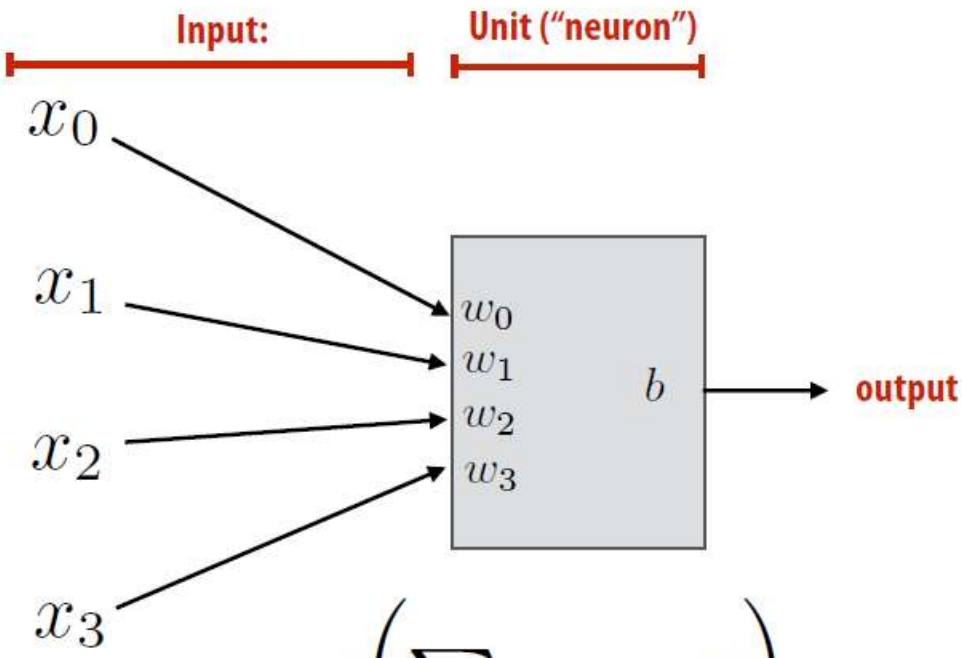
# CONTENT

- DNN components
- Parallelizing DNNs
- An extra (but a common) technique: layer fusion

# DEEP NEURAL NETWORK

## A basic unit:

**Unit with *n* inputs described by *n+1* parameters (weights + bias)**




$$f\left(\sum_i x_i w_i + b\right)$$

**Example: rectified linear unit (ReLU)**

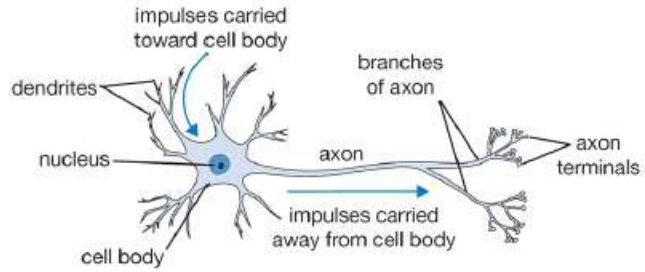
$f(x) = max(0, x)$

## Basic computational interpretation:

**It is just a circuit!**

## Biological inspiration:

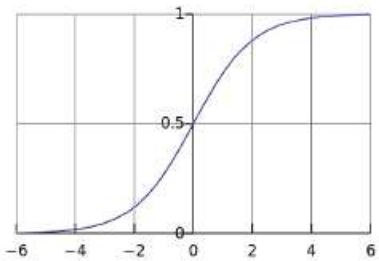
**unit output corresponds loosely to activation of neuron**




## Machine learning interpretation:

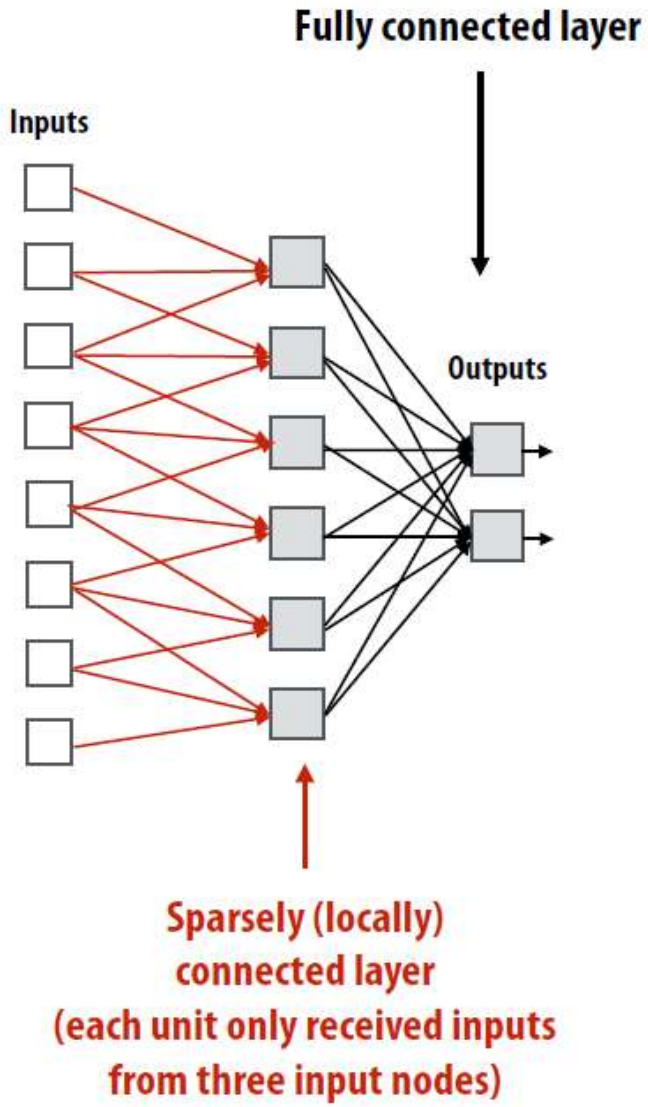
**binary classifier: interpret output as the probability of one class**

$$f(x) = \frac{1}{1 + e^{-x}}$$



# DEEP NEURAL NETWORK

- Two building blocks for deep learning



# 2D CONVOLUTION

**IN**

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 7 | 8 | 9 | 10 | 11 | 12 | 13 |

**OUT**

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | |
| | 93 | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

**Filter Mask**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 3 | 4 |
| 3 | 4 | 5 |

$*$

| | | |
|---|---|---|
| 1 | 4 | 9 |
| 4 | 9 | 16 |
| 9 | 16 | 25 |

$+$

- Usually normalized, but let's forget it for now

$$out[x,y] = in[x,y] * F = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} F[k,j]in[x-k,y-l]$$

# 2D CONVOLUTION

**IN**

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 7 | 8 | 9 | 10 | 11 | 12 | 13 |

**OUT**

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | |
| | 93 | 120 | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

**Filter Mask**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 3 | 4 |
| 3 | 4 | 5 |

$*$

| | | |
|---|---|---|
| 2 | 6 | 12 |
| 6 | 12 | 20 |
| 12 | 20 | 30 |

$+$

- Usually normalized, but let's forget it for now

$$out[x,y] = in[x,y] * F = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} F[k,j]in[x-k,y-l]$$

# 2D CONVOLUTION

**IN**

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 7 | 8 | 9 | 10 | 11 | 12 | 13 |

**OUT**

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | |
| | 93 | 120 | 174 | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

**Filter Mask**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 3 | 4 |
| 3 | 4 | 5 |

$*$

| | | |
|---|---|---|
| 4 | 10 | 18 |
| 10 | 18 | 28 |
| 18 | 28 | 40 |

$+$

- Usually normalized, but let's forget it for now

$$out[x,y] = in[x,y] * F = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} F[k,j]in[x-k,y-l]$$

# TEMPLATE MATCHING

**IN**

| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

**F**

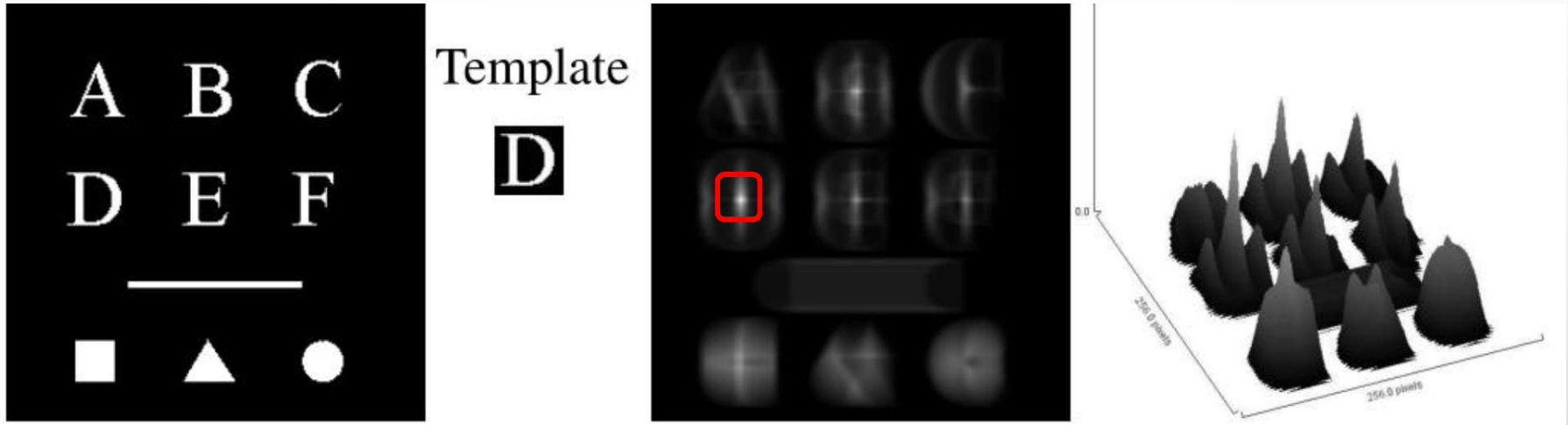
| | | |
|---|---|---|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

- You can think of a filter as a pattern 'detector'

| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | -2 | 2 | -2 |
| 0 | 0 | 0 | -3 | 3 | -3 |
| 0 | 0 | 0 | -2 | 2 | -2 |
| 0 | 0 | 0 | -1 | 1 | -1 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

- This filter detects a vertical strip

# TEMPLATE MATCHING



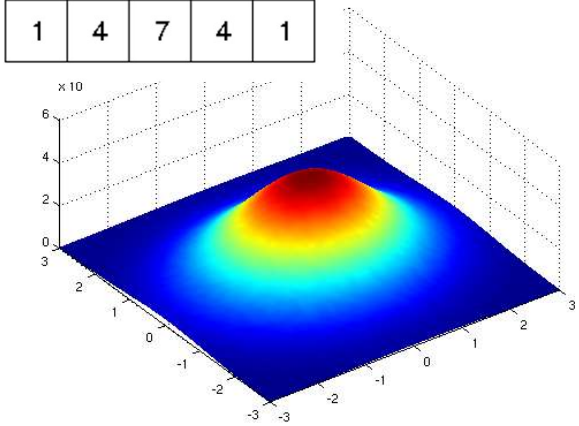

- **Image classification**: what kind of object is it?
- **Object detection**: is some object in the image, if yes, where?
- Of course, with many problems
  - Different size
  - Different rotational orientation

# BLURRING

- We have seen that filters can blur images
  - Gaussian filter
    - Value at the center is more important!
  - Median blur
    - Not a convolutional filter, but useful
  - Simple averaging also works

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$



Original

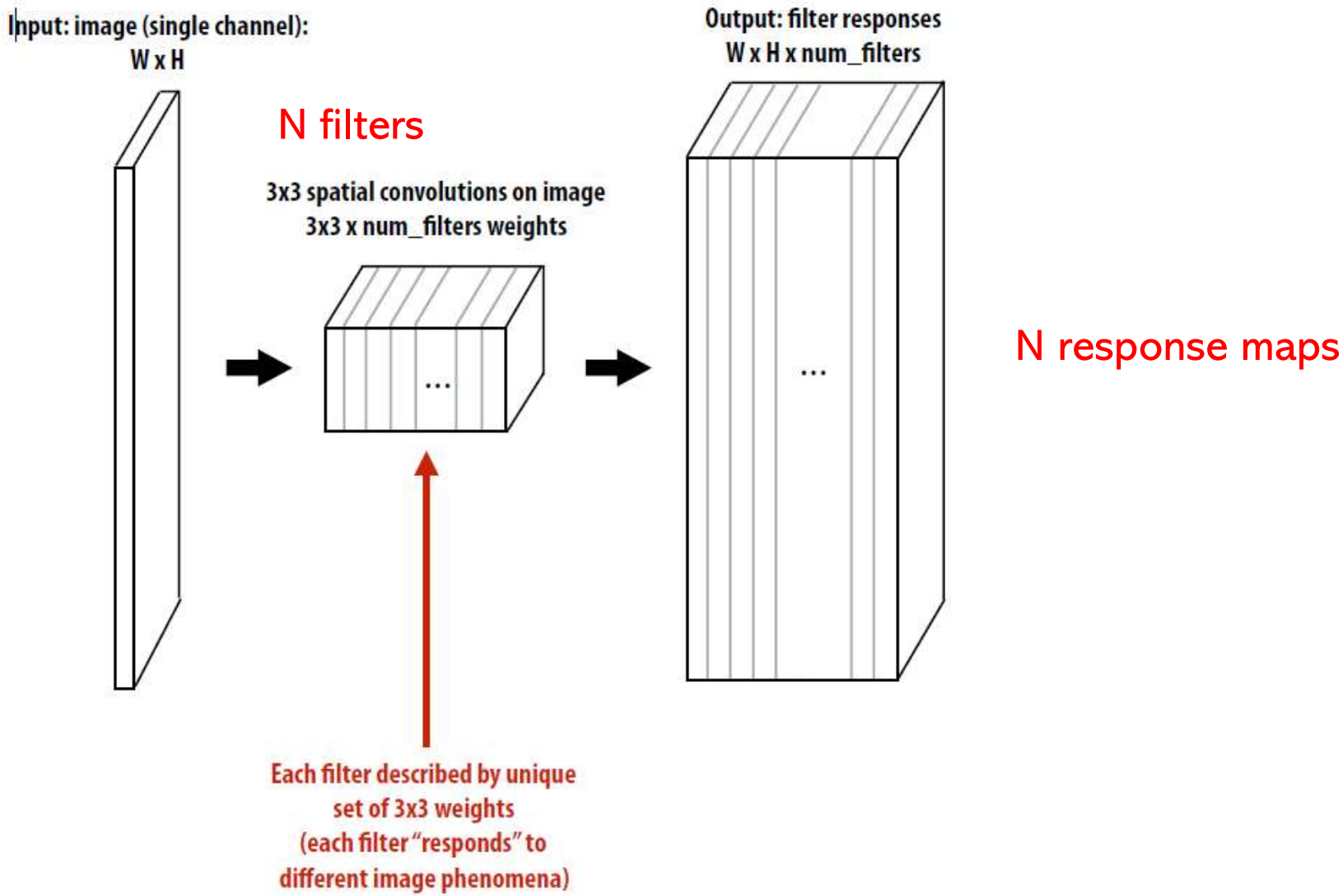
Gaussian Blur

Median Blur



https://datacarpentry.org/image-processing/06-blurring/

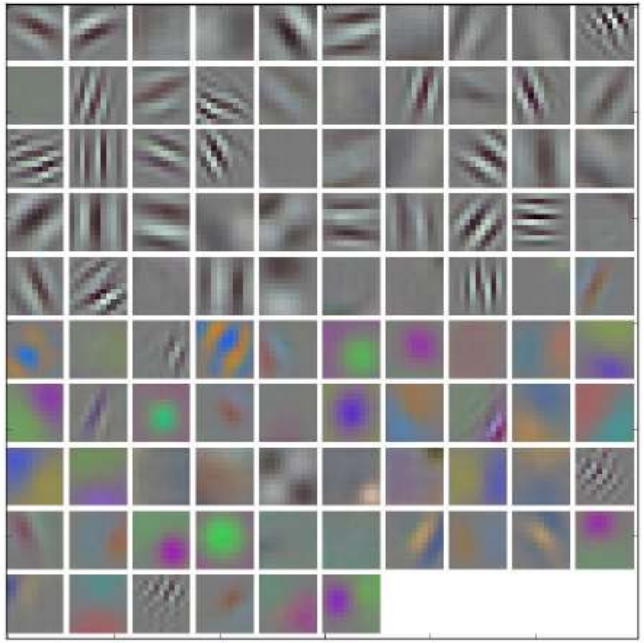
# USE SO MANY FILTERS



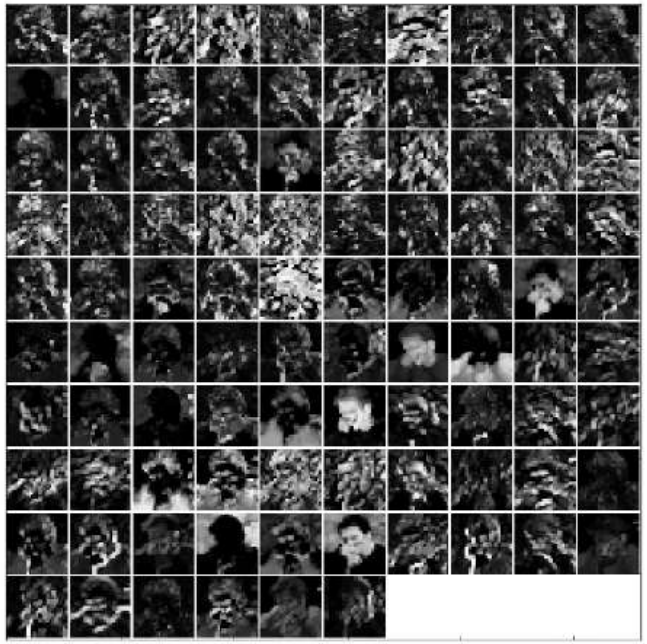
# HAVE SO MANY FILTERS

Input RGB image (W x H x 3)

96 11x11x3 filters
(operate on RGB)



96 responses (normalized)



# ACTIVATION FUNCTION (RELU)

**IN**

| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

- You can think of a filter as a pattern 'detector'

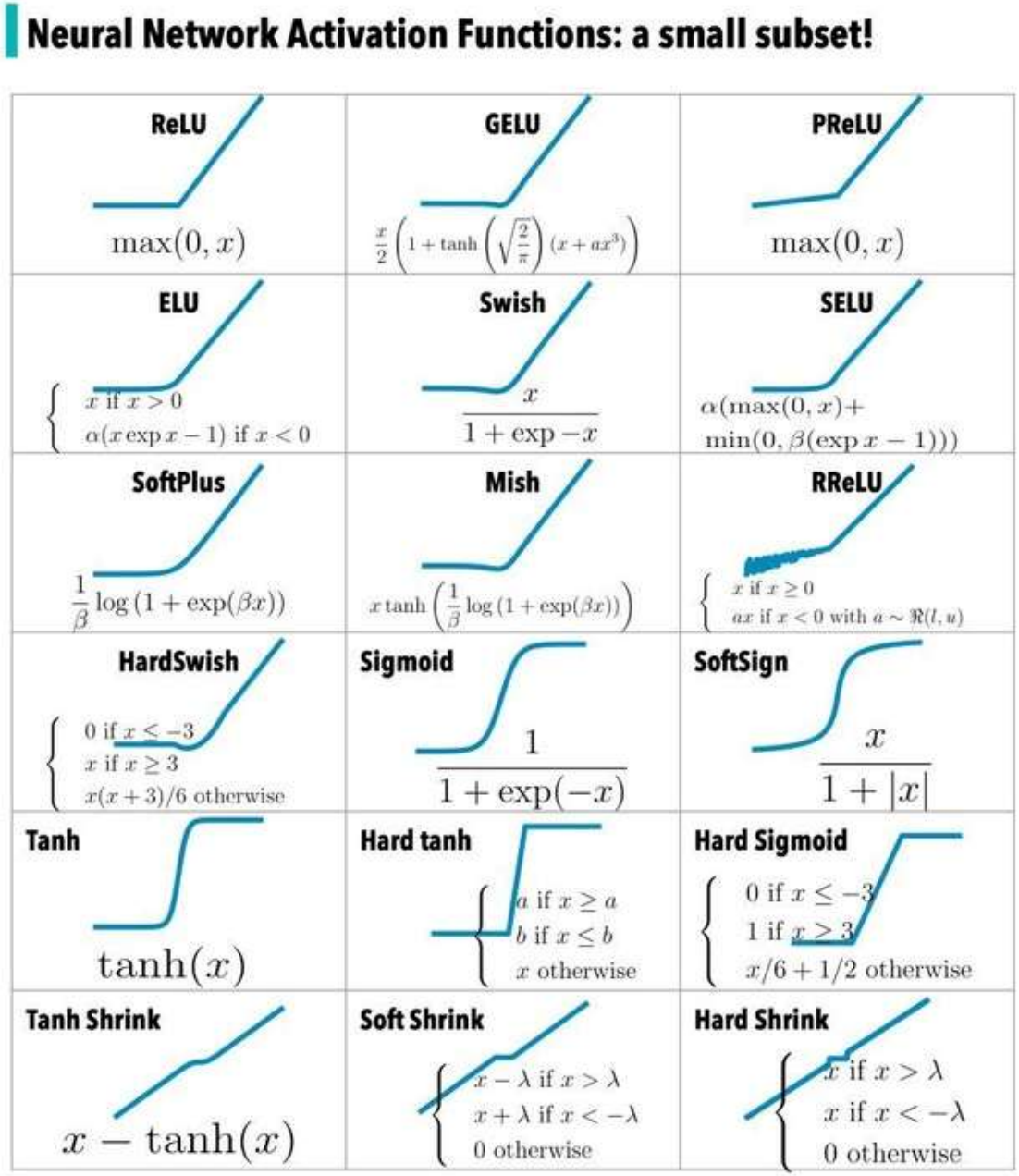
| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 2 | 0 |
| 0 | 0 | 0 | 0 | 3 | 0 |
| 0 | 0 | 0 | 0 | 2 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

- Put all negatives to ‘0’
  - ReLU
- Called non-linearity, or activation function

- This filter detects a vertical strip

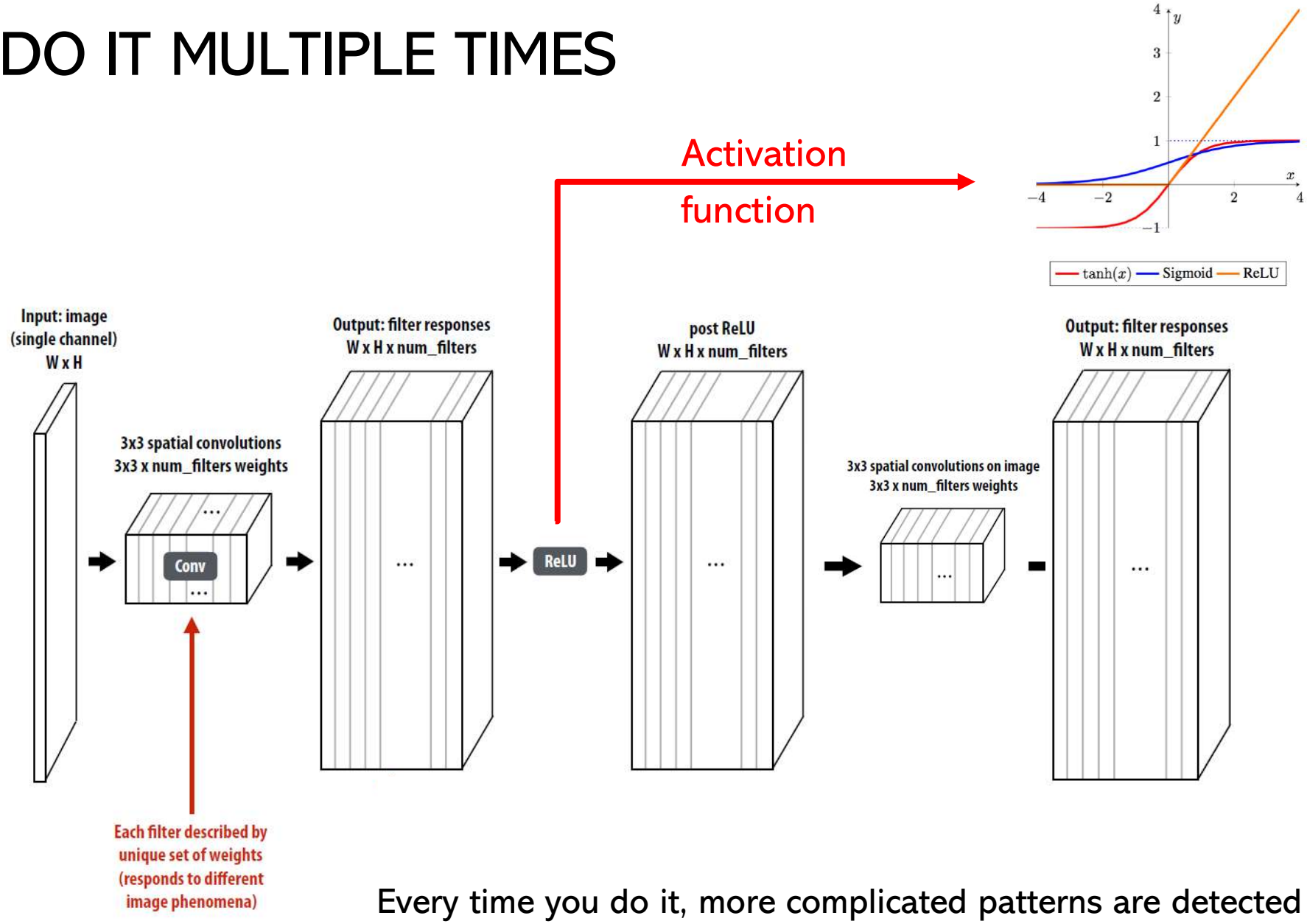
# ACTIVATION FUNCTIONS

- Many kinds of activation functions exist
- We will use ReLU, Sigmoid, and Swish (SiLU) in our project
- Each with their pros and cons, but it's out of scope to us
- We will just implement according to their equations
- They are all element-wise operations



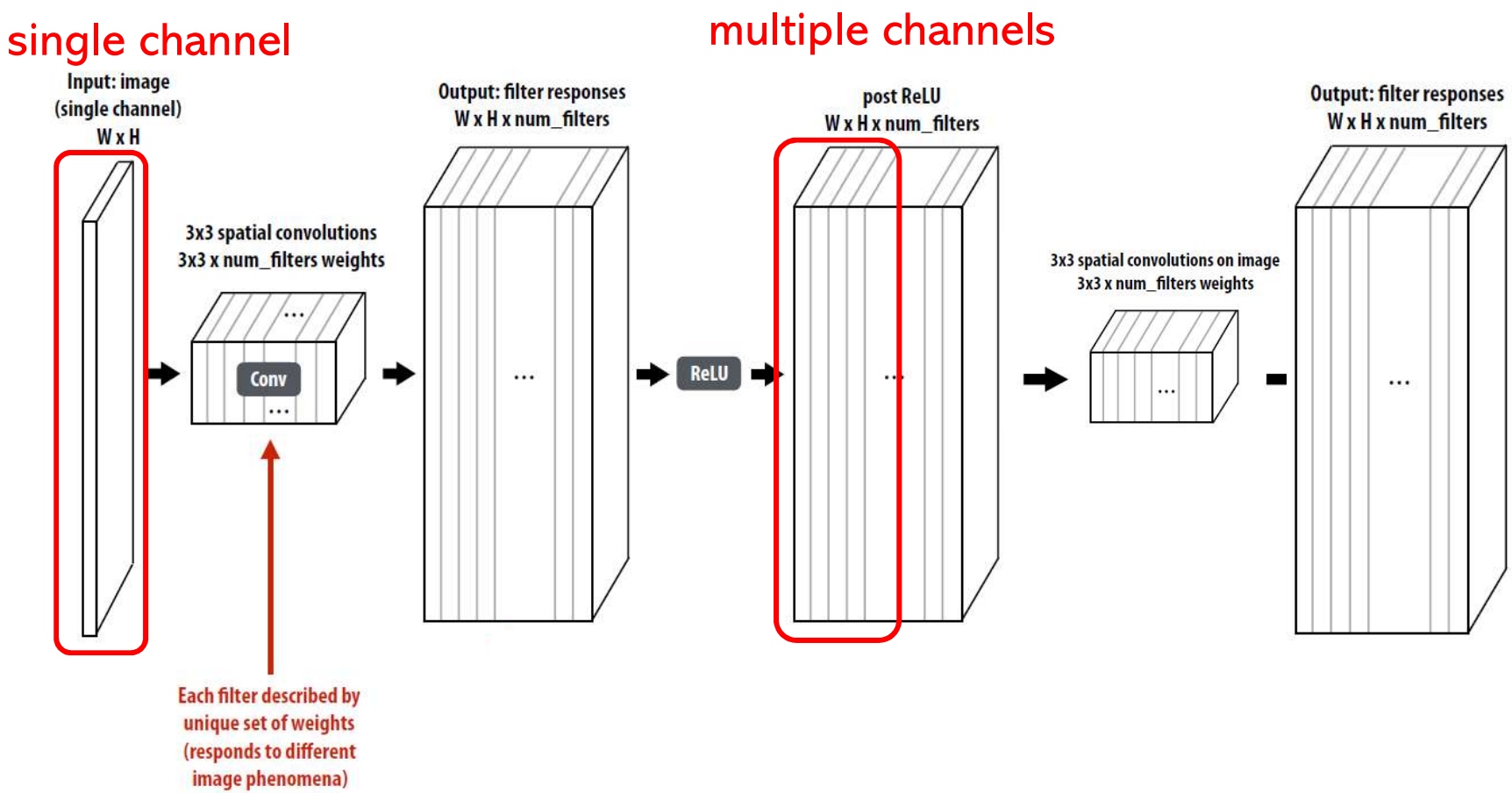

https://www.linkedin.com/pulse/activation-functions-heba-al-haddad/

# DO IT MULTIPLE TIMES



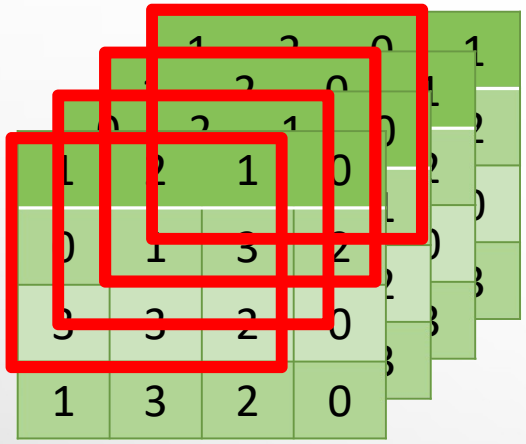

Every time you do it, more complicated patterns are detected

# DO IT MULTIPLE TIMES

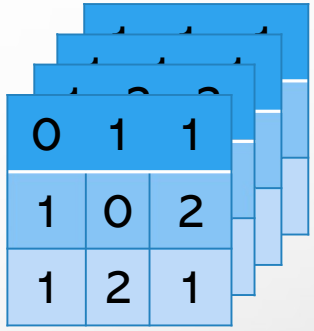



→ We need to generalize 2D convolution to multiple channels

# When the input image has four channels



**input image**
(four channels)




**3x3 filter mask**
(must have four channels)

- Multiply-add 3x3x4 values for a single output pixel

# C=3 H=4 W=4 K=3

X[0,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 1 | 1 | 3 | 2 |
| 0 | 2 | 2 | 0 |
| 2 | 1 | 0 | 3 |

X[1,_, _]

| | | | |
|---|---|---|---|
| 0 | 2 | 1 | 0 |
| 0 | 3 | 2 | 1 |
| 1 | 1 | 0 | 2 |
| 2 | 1 | 0 | 3 |

X[2,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 1 | 0 |
| 0 | 1 | 3 | 2 |
| 3 | 3 | 2 | 0 |
| 1 | 3 | 2 | 0 |

**X[C,H,W]**

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 2 | 1 | 0 |

W[0,0,_, _]

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 1 | 1 | 0 |
| 3 | 0 | 1 |

W[0,1,_, _]

| | | |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 0 | 2 |
| 1 | 2 | 1 |

W[0,2,_, _]

**W[M,C,K1,K2]**

3+13+2

| | |
|---|---|
| 18 | ? |
| ? | ? |

Y[0,_, _]

c=0 n=0 m=0

**Y[M,H,W]**

# C=3 H=4 W=4 K=3

X[0,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 1 | 1 | 3 | 2 |
| 0 | 2 | 2 | 0 |
| 2 | 1 | 0 | 3 |

X[1,_, _]

| | | | |
|---|---|---|---|
| 0 | 2 | 1 | 0 |
| 0 | 3 | 2 | 1 |
| 1 | 1 | 0 | 2 |
| 2 | 1 | 0 | 3 |

X[2,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 1 | 0 |
| 0 | 1 | 3 | 2 |
| 3 | 3 | 2 | 0 |
| 1 | 3 | 2 | 0 |

**X[C,H,W]**

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 2 | 1 | 0 |

W[0,0,_, _]

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 1 | 1 | 0 |
| 3 | 0 | 1 |

W[0,1,_, _]

| | | |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 0 | 2 |
| 1 | 2 | 1 |

W[0,2,_, _]

**W[M,C,K1,K2]**

7+3+3

| | |
|---|---|
| 31 | ? |
| ? | ? |

Y[0,_, _]

c=1 n=0 m=0

**Y[M,H,W]**

# C=3 H=4 W=4 K=3

X[0,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 1 | 1 | 3 | 2 |
| 0 | 2 | 2 | 0 |
| 2 | 1 | 0 | 3 |

X[1,_, _]

| | | | |
|---|---|---|---|
| 0 | 2 | 1 | 0 |
| 0 | 3 | 2 | 1 |
| 1 | 1 | 0 | 2 |
| 2 | 1 | 0 | 3 |

X[2,_, _]

| | | | |
|---|---|---|---|
| 1 | 2 | 1 | 0 |
| 0 | 1 | 3 | 2 |
| 3 | 3 | 2 | 0 |
| 1 | 3 | 2 | 0 |

**X[C,H,W]**

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 2 | 1 | 0 |

W[0,0,_, _]

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 1 | 1 | 0 |
| 3 | 0 | 1 |

W[0,1,_, _]

| | | |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 0 | 2 |
| 1 | 2 | 1 |

W[0,2,_, _]

**W[M,C,K1,K2]**

3+6+11

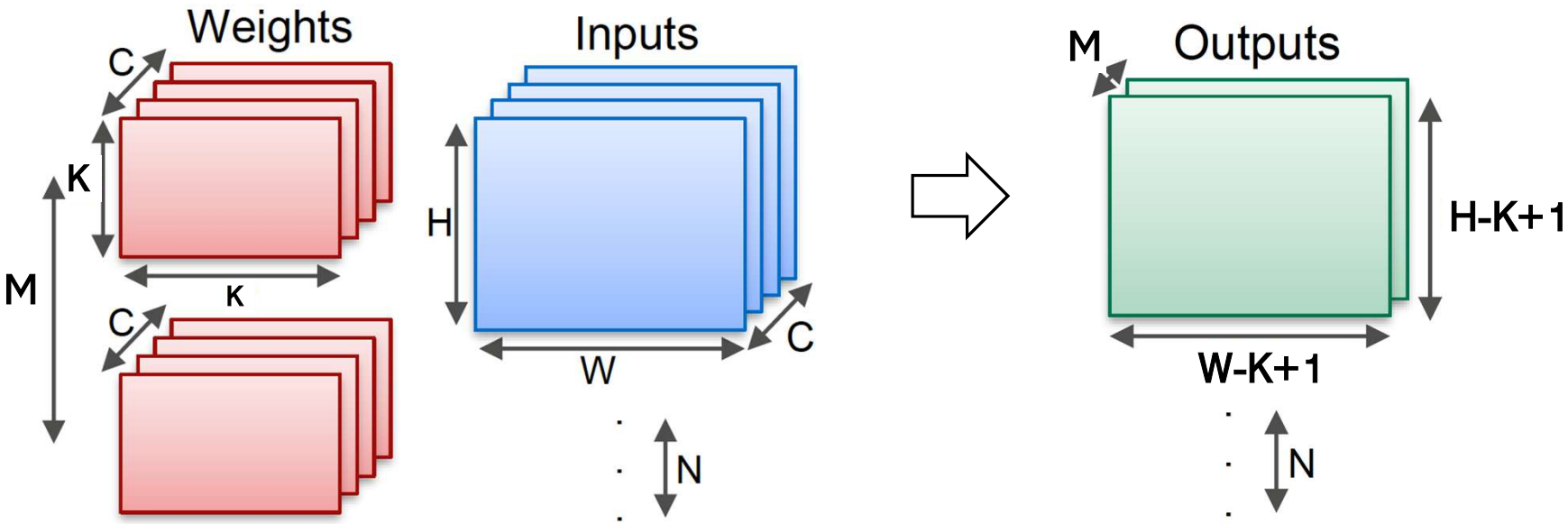
| | |
|---|---|
| 51 | ? |
| ? | ? |

Y[0,_, _]

c=2 n=0 m=0

**Y[M,H,W]**

# 7 DIMENSIONS OF CNN




- 7-nested loop : $N \rightarrow M \rightarrow H \rightarrow W \rightarrow C \rightarrow K_1 \rightarrow K_2$
- Minibatch (N), output channels (M), input height (H), input width (W), input feature maps (C), weight height ($K_1$), weight width ($K_2$)
- Usually called [Number of feature maps = number of channels]
- Minibatch (N) = Number of images to process together

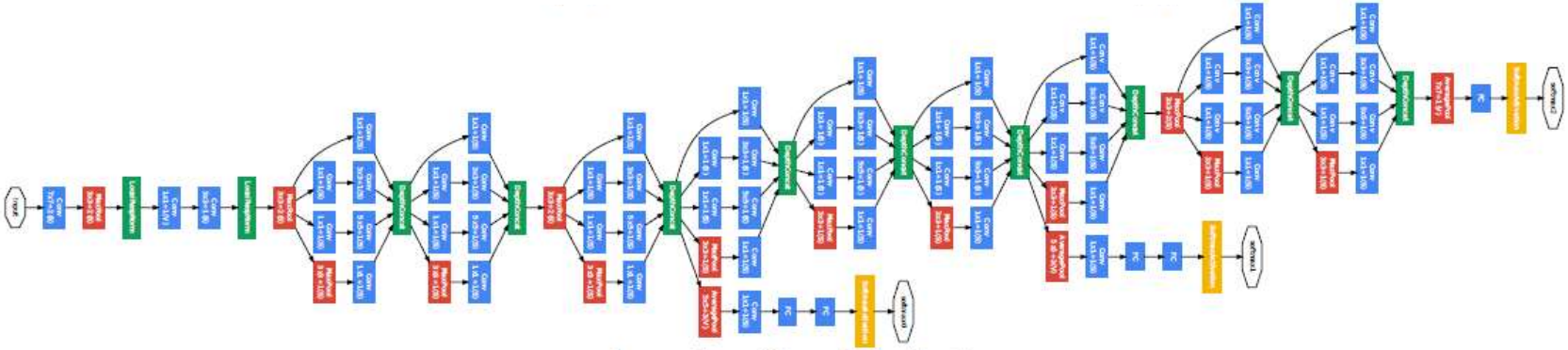
# A CONVOLUTIONAL LAYER

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                                                float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)            //for each input image in a minibatch
     for(int m = 0;  m < M;  m++)        // for each output feature map
       for(int h = 0; h < H_out; h++)          // for each output element
         for(int w = 0; w < W_out; w++) {
            Y[n, m, h, w] = 0;
            for(int c = 0;  c < C; c++)            // sum over all input feature maps
              for(int p = 0; p < K; p++)         // KxK  filter
                 for(int q = 0; q < K; q++)
                     Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
         }
}
```

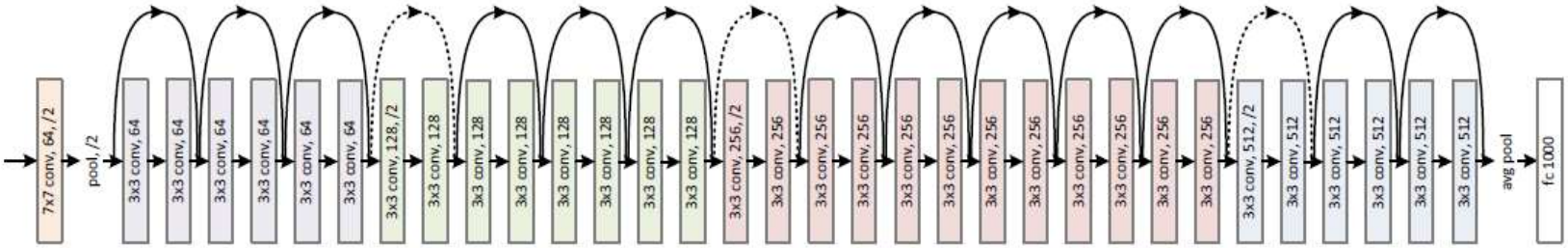
# A CONVOLUTIONAL LAYER

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                                                float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)            //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)        // for each output feature map
      for(int h = 0; h < H_out; h++)          // for each output element
        for(int w = 0; w < W_out; w++) {
           Y[n, m, h, w] = 0;
           for(int c = 0;  c < C; c++)            // sum over all input feature maps
             for(int p = 0; p < K; p++)         // KxK  filter
                for(int q = 0; q < K; q++)
                    Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
        }
}
```

# DO IT MANY MORE TIMES….



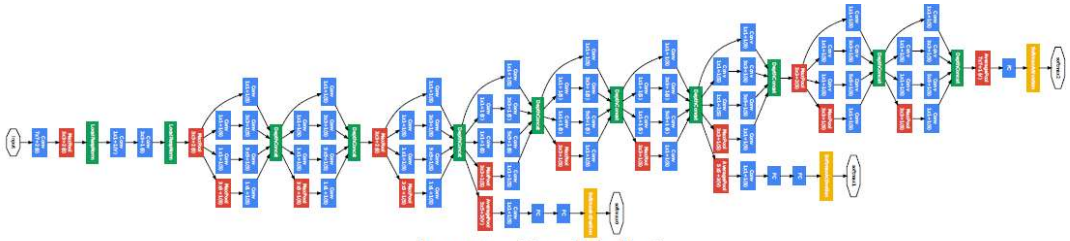
Inception (GoogleLeNet)



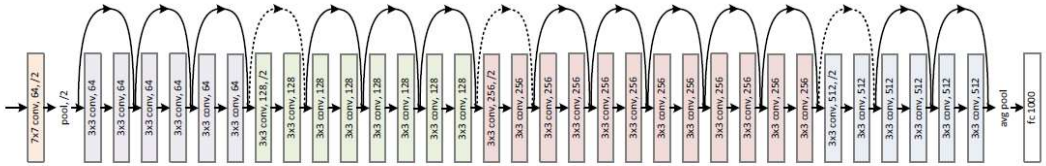

ResNet (34 layer version)

- Each layer gives you ‘higher level’ features

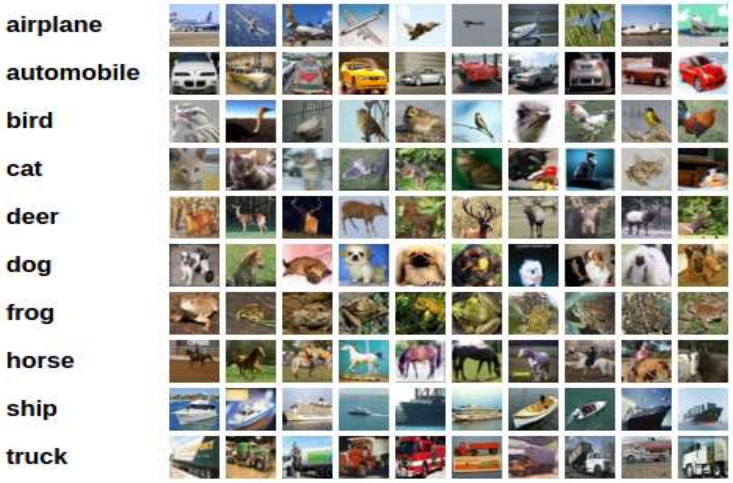
# UNTIL YOU CAN RECOGNIZE IMAGES



Inception (GoogleLeNet)




ResNet (34 layer version)

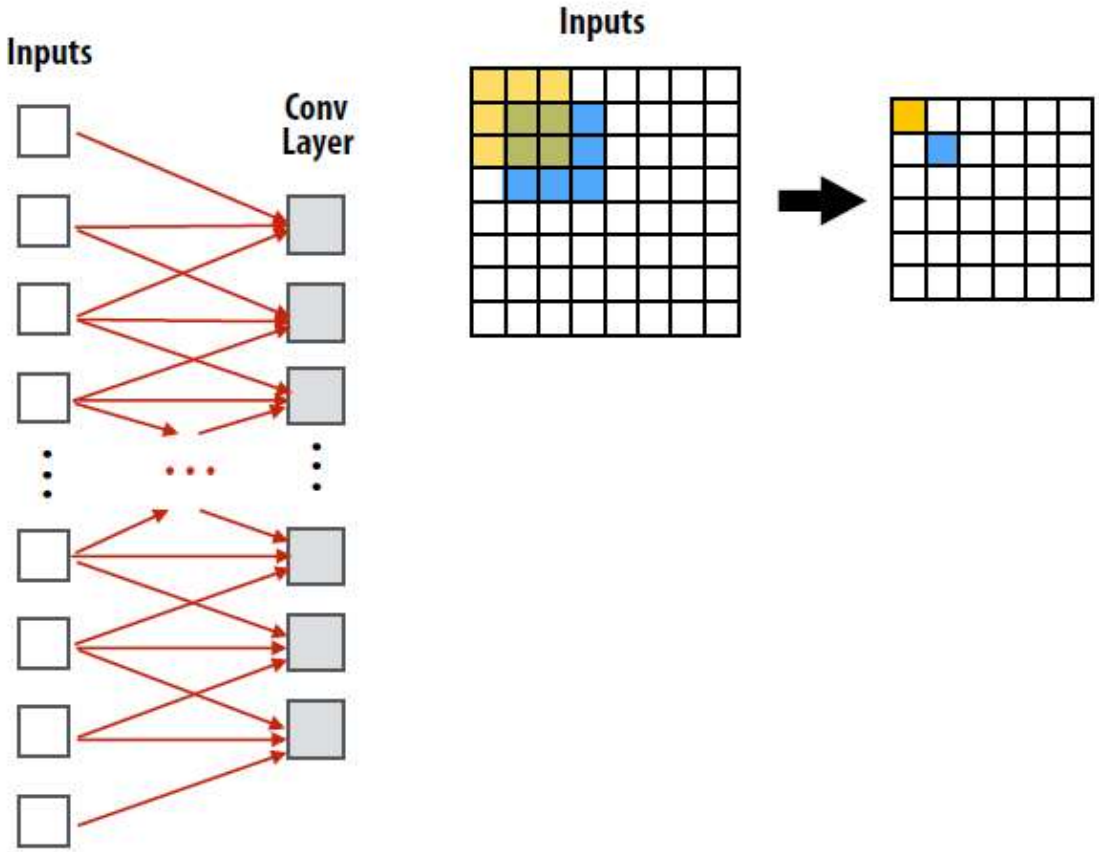



- Deciding the filter parameter values is done through “training”
- We will not care about training. Refer to other classes (e.g., Deep learning)

# SELF-STUDY ISSUES ON CONVOLUTIONAL LAYERS

- Strides
- Padding
- Dilation
- Group convolution
- Depthwise convolution
- Layerwise convolution
- Batch normalization
- Group normalization
- Layer normalization

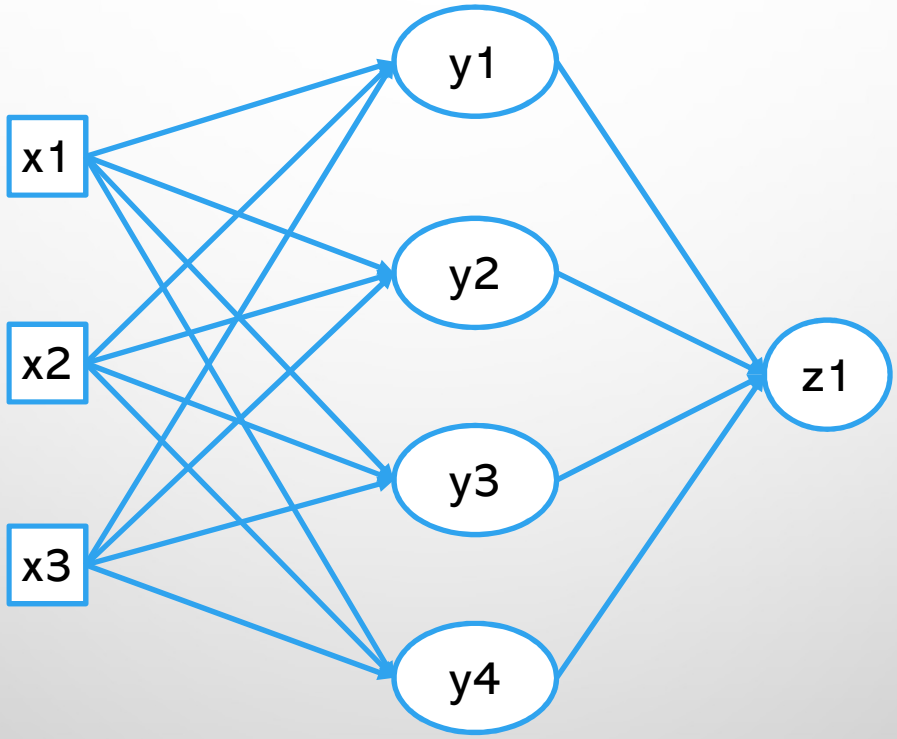
# CONVOLUTIONAL LAYER IS A SPARSELY CONNECTED LAYER




**Convolutional layer: locally connected AND all units in layer share the same parameters (same weights + same bias):**

# ..AS OPPOSED TO FULLY CONNECTED LAYERS

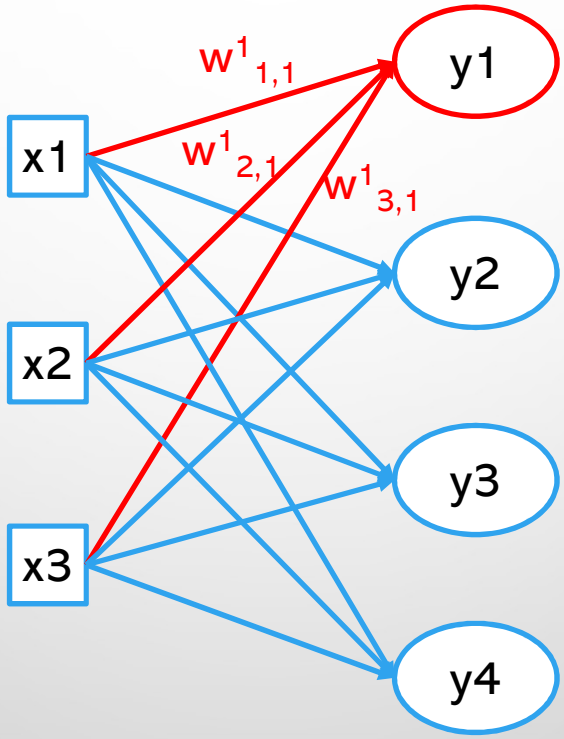
- The weight parameters are multiplied to the inputs




A “shallow” neural network with only one hidden layer (nodes y1, y2, y3, y4), three inputs and one output.

# FULLY CONNECTED LAYERS

- The weight parameters are multiplied to the inputs

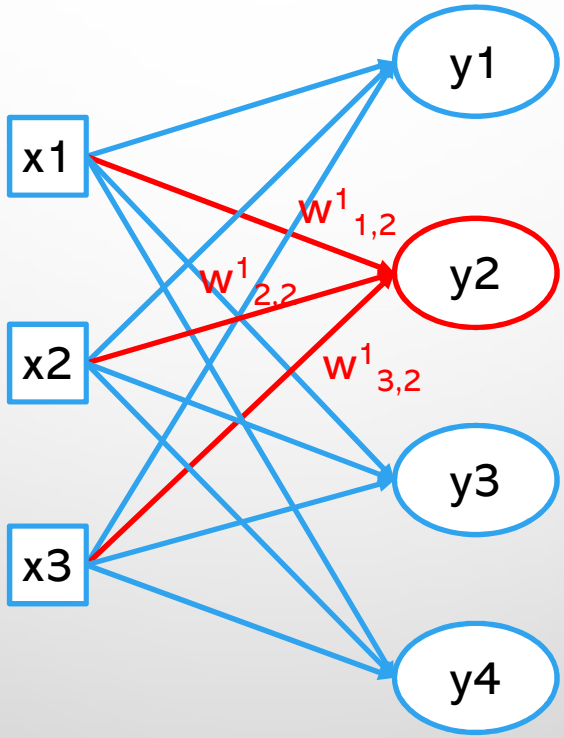



$$y_1 = \sum_{i=1}^{3} w_{i1}^{1} x_i$$

The intermediate value (=activation) y1
is connected to all inputs (x1, x2, x3)
→ fully connected

# FULLY CONNECTED LAYERS

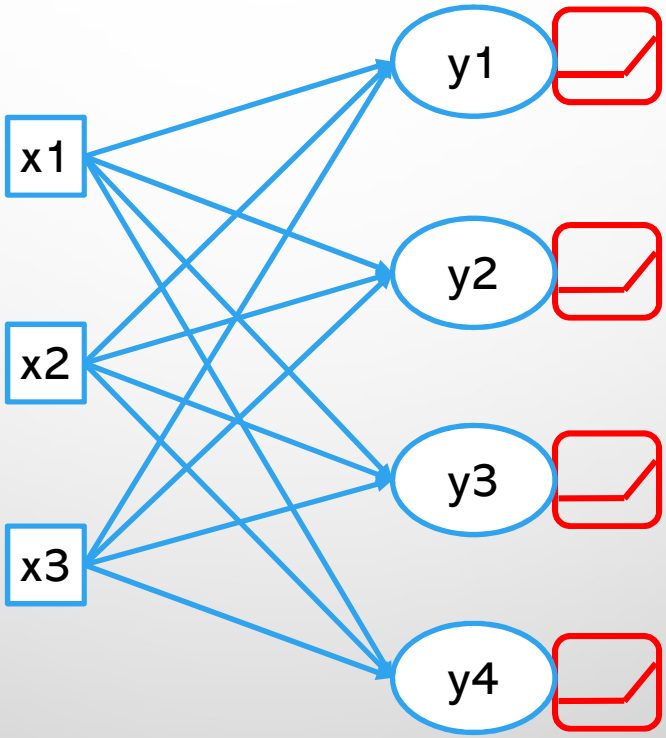
- The weight parameters are multiplied to the inputs




$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$

The intermediate value (=activation) y2
is connected to all inputs (x1, x2, x3)
→ fully connected

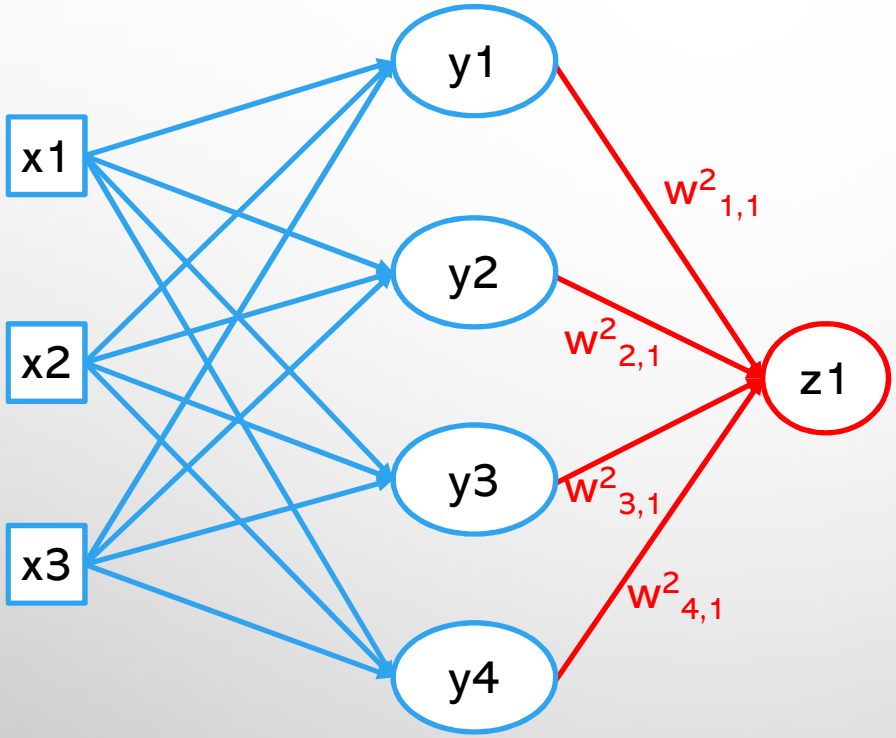
# FULLY CONNECTED LAYERS






- Activation functions are applied to each intermediate value

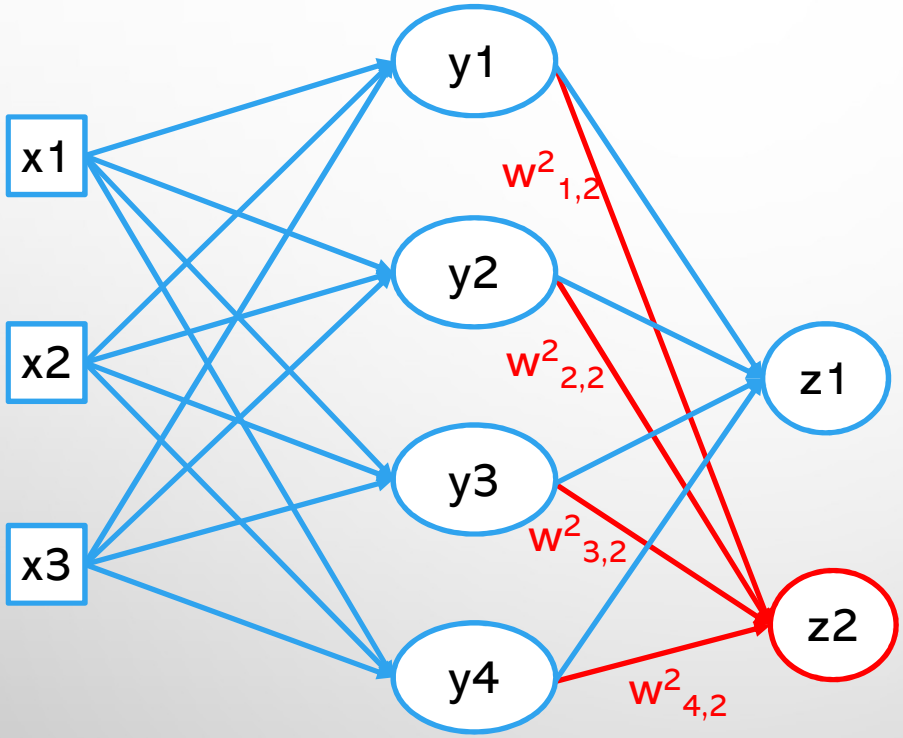
# FULLY CONNECTED LAYERS




$$z_1 = \sum_{i=1}^{4} w_{i1}^2 y_i$$

- Next fully connected layer does the same

# FULLY CONNECTED LAYERS



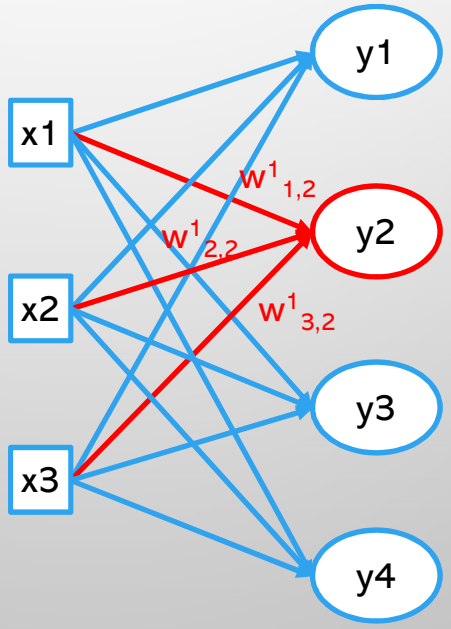

→ confidence for Dog

→ confidence for Cat

- Adding another intermediate output requires adding four more parameters

# A FULLY CONNECTED LAYER

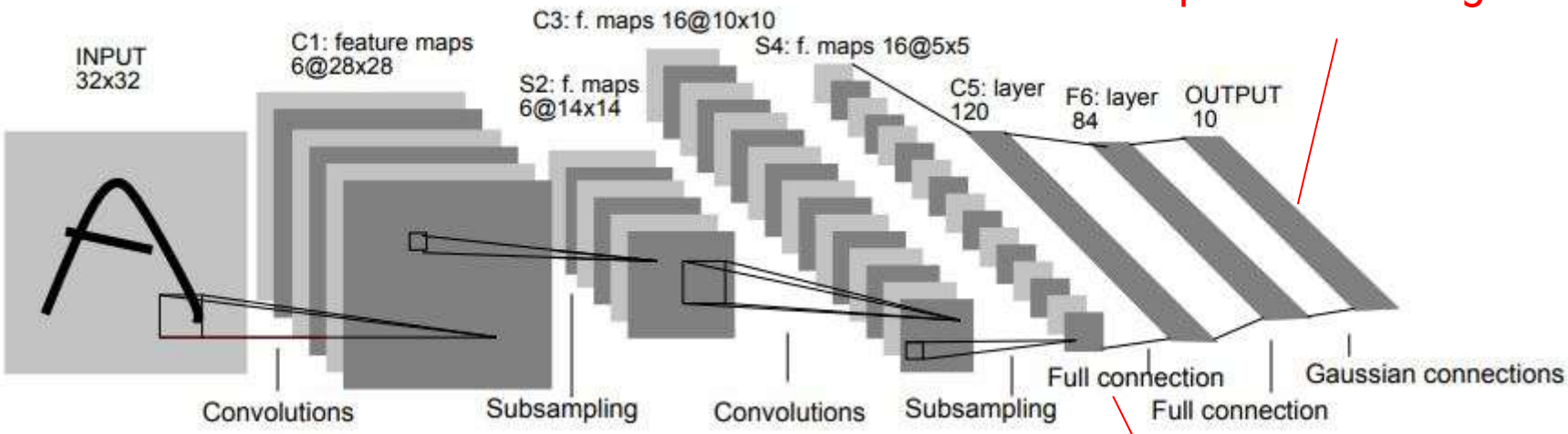
```
void fcLayer_forward(int N, int M, int C,
                                    float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)         // for each output element
      Y[n, m] = 0;
      for(int c = 0;  c < C; c++)        // sum over all input elements
        Y[n, m] += X[n, c] * W[m, c];
        }
}
```




$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$

# LENET-5, A CONVOLUTIONAL NEURAL NETWORK FOR HAND-WRITTEN DIGIT RECOGNITION.

10 output values represent 0~9 digits



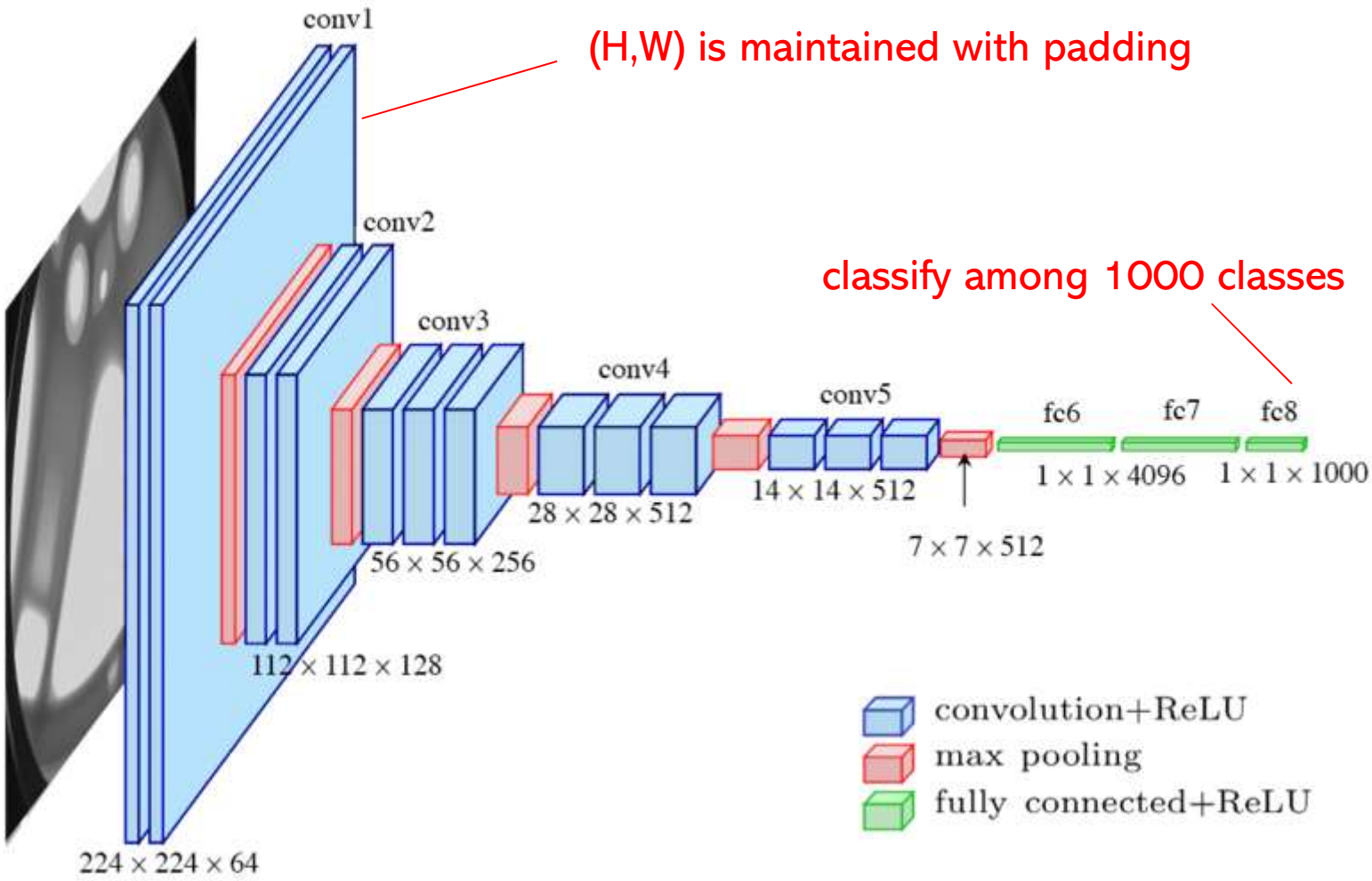

5x5 convolution reduces the (H,W) dimension

switches from conv layers to fc layers

2x2 pooling reduces the (H,W) dimension to half

# MAKE LARGER NETWORKS

- VGGNet (ImageNet challenge 2014 2nd place)

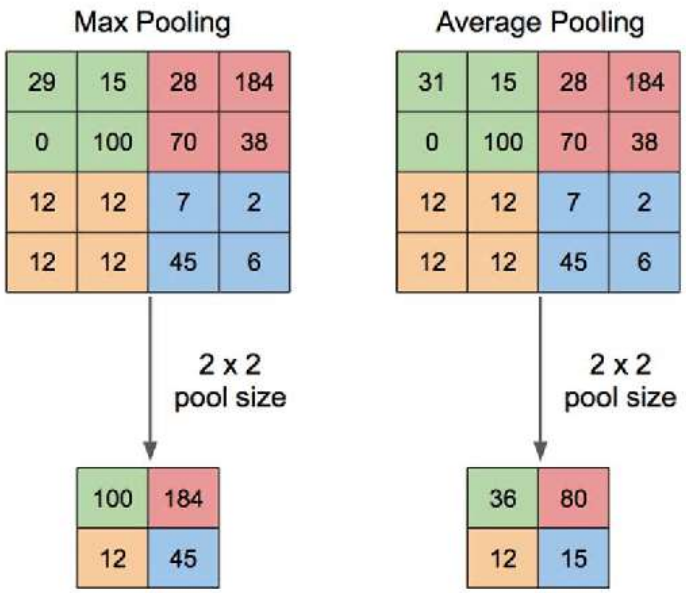





https://clever-keller-127e62.netlify.app/blog/what-is-vggnet/clip_image001.png

# POOLING (SO SIMPLE)

```
void poolingLayer_forward(int M, int H, int W, int K, float* Y, float* S)
{
  for(int m = 0;  m < M;  m++)          // for each output feature maps
    for(int h = 0; h < H/K; h++)        // for each output element
      for(int w = 0; w < W/K; w++) {
        S[m, h, w] = 0.;
        for(int p = 0; p < K; p++) {        // loop over KxK input samples
          for(int q = 0; q < K; q++)
            S[m, h, w] += Y[m, K*h + p, K*w + q] /(K*K);  // avg pooling
        }
   }
}
```

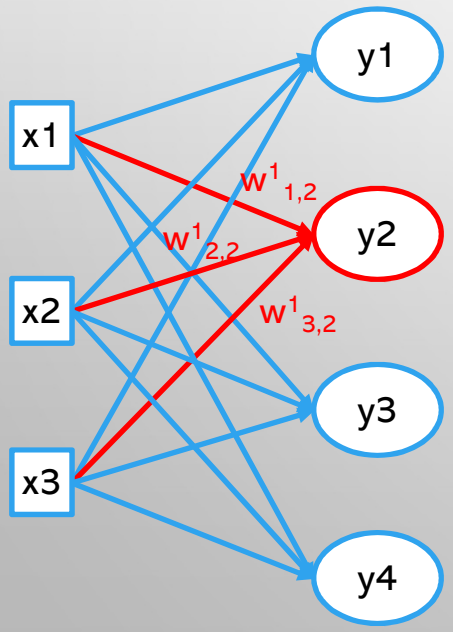


# CONTENT

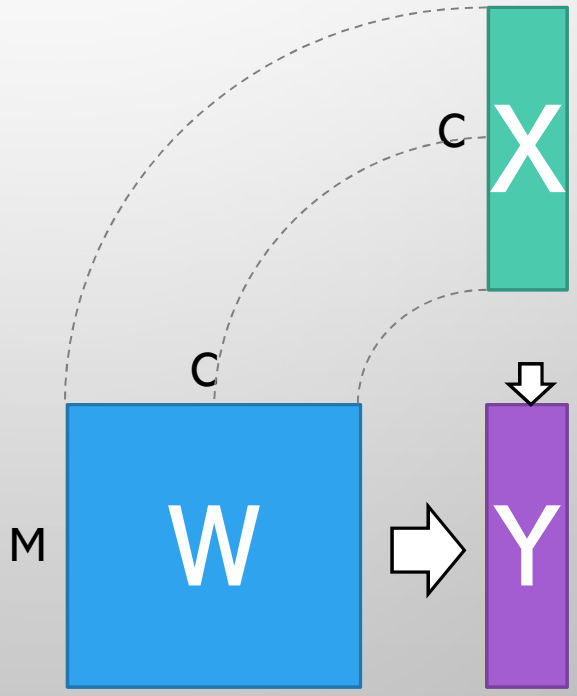
- DNN components
- Parallelizing DNNs
- An extra (but a common) technique: layer fusion

# FC LAYER = MATRIX MULTIPLICATION

```
void fcLayer_forward(int N, int M, int C,
                                    float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)         // for each output element
      Y[n, m] = 0;
      for(int c = 0;  c < C; c++)        // sum over all input elements
        Y[n, m] += X[n, c] * W[m, c];
        }
}
```

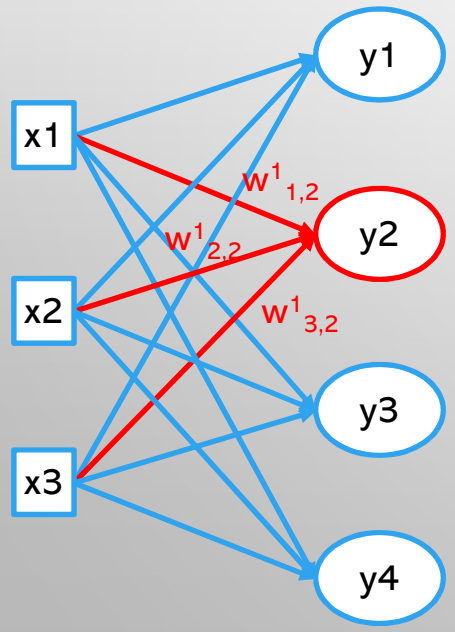



$$y_2 = \sum_{i=1}^{3} w^1_{i2} x_i$$

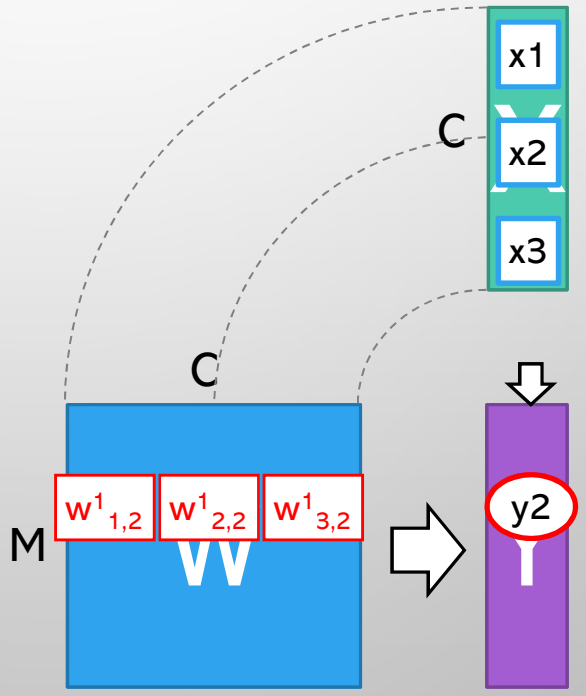


# FC LAYER = MATRIX MULTIPLICATION

```
void fcLayer_forward(int N, int M, int C,
                                    float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)         // for each output element
      Y[n, m] = 0;
      for(int c = 0;  c < C; c++)        // sum over all input elements
        Y[n, m] += X[n, c] * W[m, c];
        }
}
```

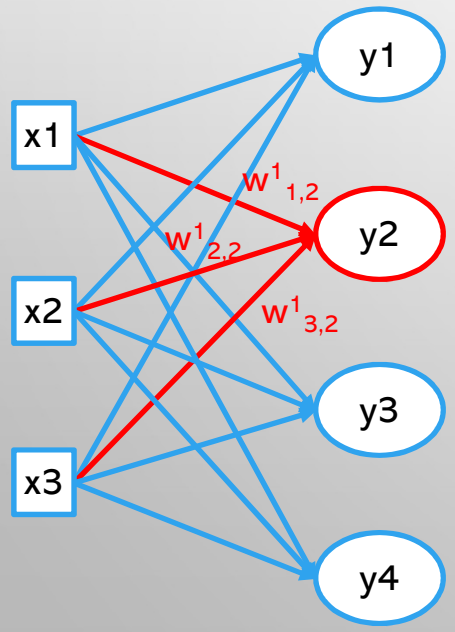



$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$

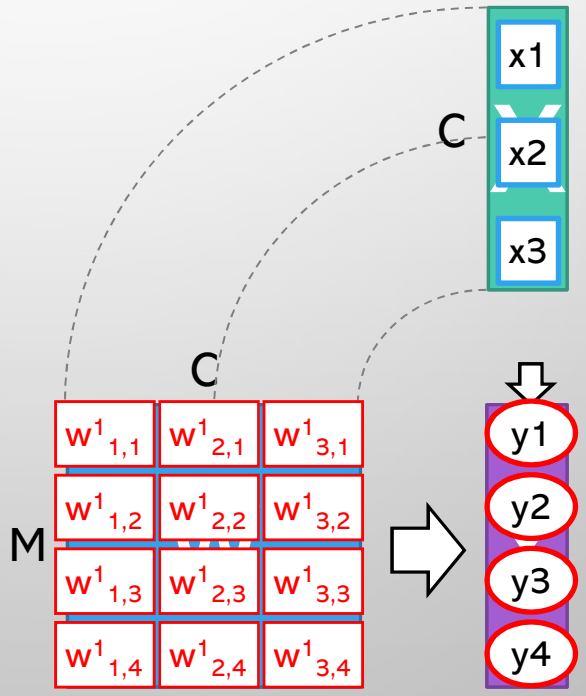


# FC LAYER = MATRIX MULTIPLICATION

```
void fcLayer_forward(int N, int M, int C,
                                    float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)         // for each output element
      Y[n, m] = 0;
      for(int c = 0;  c < C; c++)        // sum over all input elements
        Y[n, m] += X[n, c] * W[m, c];
        }
}
```

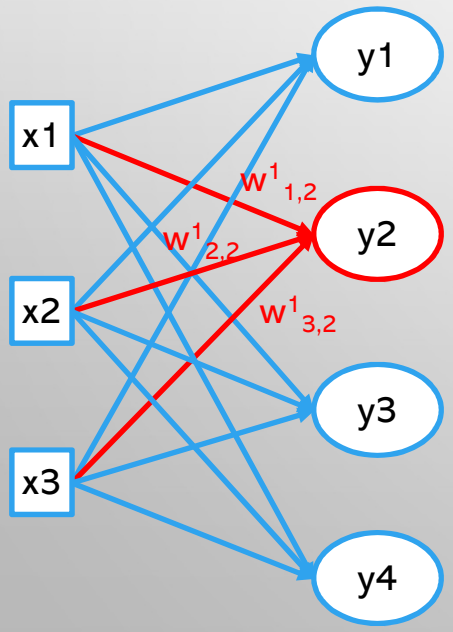



$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$

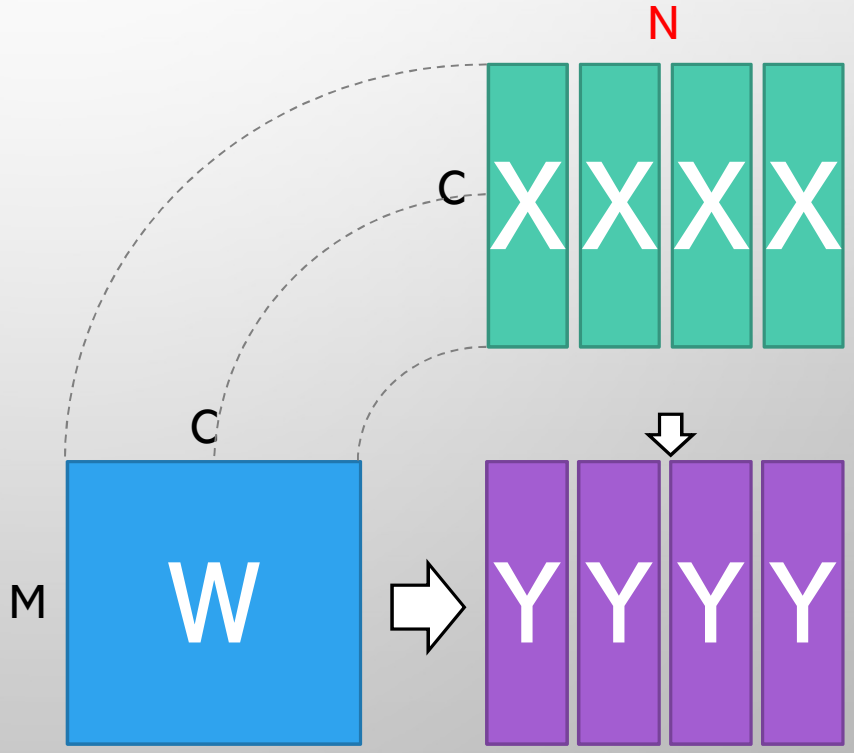


# FC LAYER = MATRIX MULTIPLICATION

```
void fcLayer_forward(int N, int M, int C,
                                    float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
    for(int m = 0;  m < M;  m++)         // for each output element
      Y[n, m] = 0;
      for(int c = 0;  c < C; c++)        // sum over all input elements
        Y[n, m] += X[n, c] * W[m, c];
        }
}
```



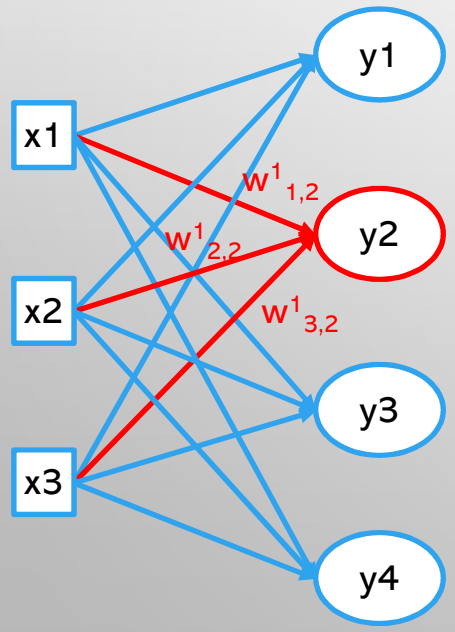

$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$



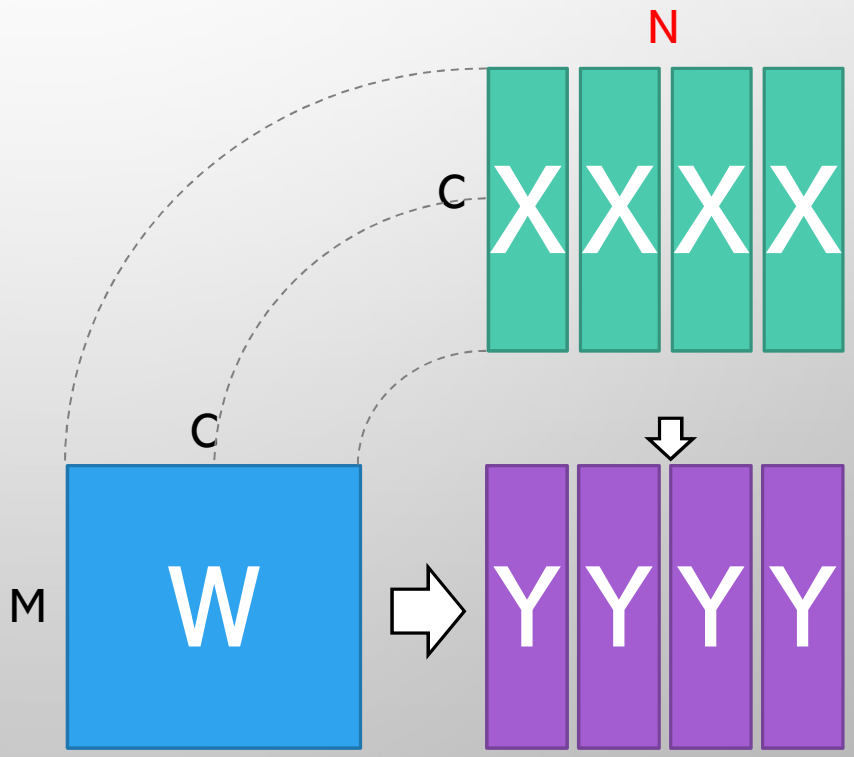
# FC LAYER = MATRIX MULTIPLICATION

PARALLELIZING FC LAYER = PARALLELIZING MATRIX MULTIPLICATION

```
void
{
  for(int n = 0;  n < N;  n++)          //for each input image in a minibatch
   for(int m = 0;  m < M;  m++)         // for each output element
    Y[n, m] = 0;
    for(int c = 0;  c < C; c++)         // sum over all input elements
     Y[n, m] += X[n, c] * W[m, c];
     }
}
```




$$y_2 = \sum_{i=1}^{3} w_{i2}^{1} x_i$$



# PARALLELIZING CONV LAYERS

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                                    float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
    for(int m = 0;  m < M;  m++)        // for each output feature map
      for(int h = 0; h < H_out; h++)          // for each output element
        for(int w = 0; w < W_out; w++) {
           Y[n, m, h, w] = 0;
           for(int c = 0;  c < C; c++)          // sum over all input feature maps
             for(int p = 0; p < K; p++)         // KxK  filter
                for(int q = 0; q < K; q++)
                    Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
        }
}
```

7 levels of loops seems easy to parallelize
especially with openmp – just add “parallel for” pragma

# PARALLELIZING CONV LAYERS

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                       float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
     for(int m = 0;  m < M;  m++)        // for each output feature map
       for(int h = 0; h < H_out; h++)          // for each output element
         for(int w = 0; w < W_out; w++) {
            Y[n, m, h, w] = 0;
            for(int c = 0;  c < C; c++)          // sum over all input feature maps
              for(int p = 0; p < K; p++)         // KxK  filter
                 for(int q = 0; q < K; q++)
                     Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
       }
}
```

- Batch-level parallelism (data parallelism)
  - Works well, but requires multiple images
  - Simple, small communication
  - Infeasible when N=1
    - Slightly more common for embedded systems

# PARALLELIZING CONV LAYERS

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                       float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
    for(int m = 0;  m < M;  m++)        // for each output feature map
      for(int h = 0; h < H_out; h++)        // for each output element
        for(int w = 0; w < W_out; w++) {
          Y[n, m, h, w] = 0;
          for(int c = 0;  c < C; c++)        // sum over all input feature maps
            for(int p = 0; p < K; p++)        // KxK  filter
              for(int q = 0; q < K; q++)
                Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
      }
}
```

- Output channel parallelism
  - Simple, small communication
  - Enough parallelism for later layers (M~=512)
  - Insufficient parallelism for earlier layers (M=3)
    - (We have 6 cores + SIMD)

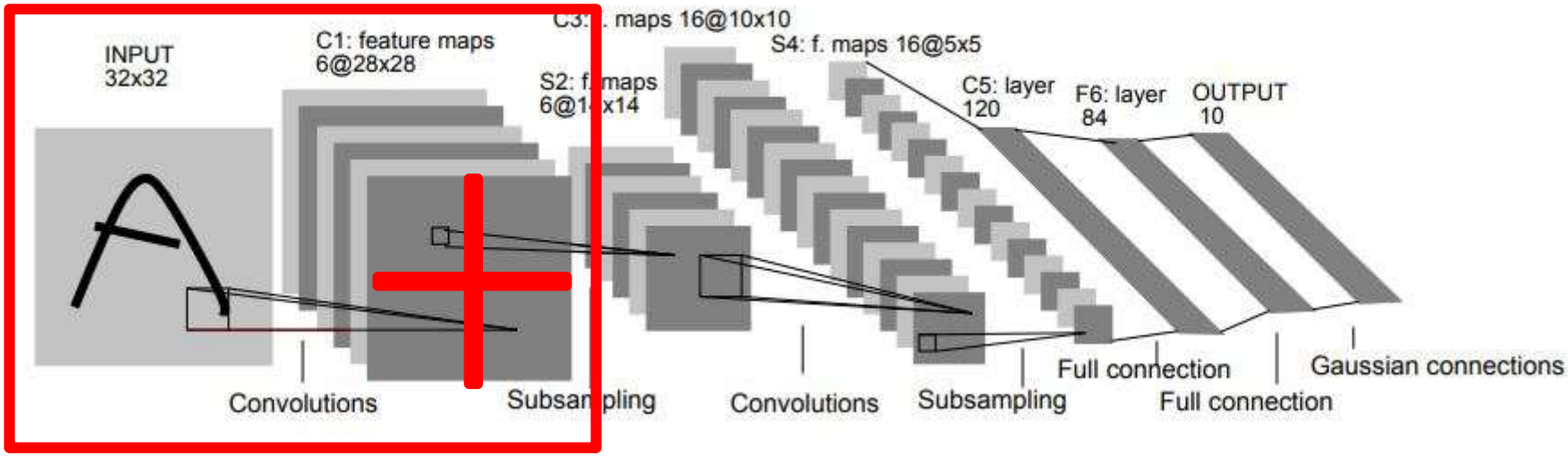
# PARALLELIZING CONV LAYERS

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                       float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
    for(int m = 0;  m < M;  m++)        // for each output feature map
     for(int h = 0; h < H_out; h++)       // for each output element
      for(int w = 0; w < W_out; w++) {
         Y[n, m, h, w] = 0;
         for(int c = 0;  c < C; c++)        // sum over all input feature maps
          for(int p = 0; p < K; p++)        // KxK  filter
           for(int q = 0; q < K; q++)
             Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
      }
}
```

- Output pixel parallelism
  - Medium-amount of communication
  - Enough parallelism for earlier layers
    - (224x224 or 32x32)
  - Insufficient parallelism for later layers
    - (5x5, 7x7)

# TILED PARALLELIZATION

- Each block computes <u>**a tile of output pixels**</u>
  - TILE_WIDTH pixels in each dimension
- This could be better than simple pixelwise parallelization
  - (why?)



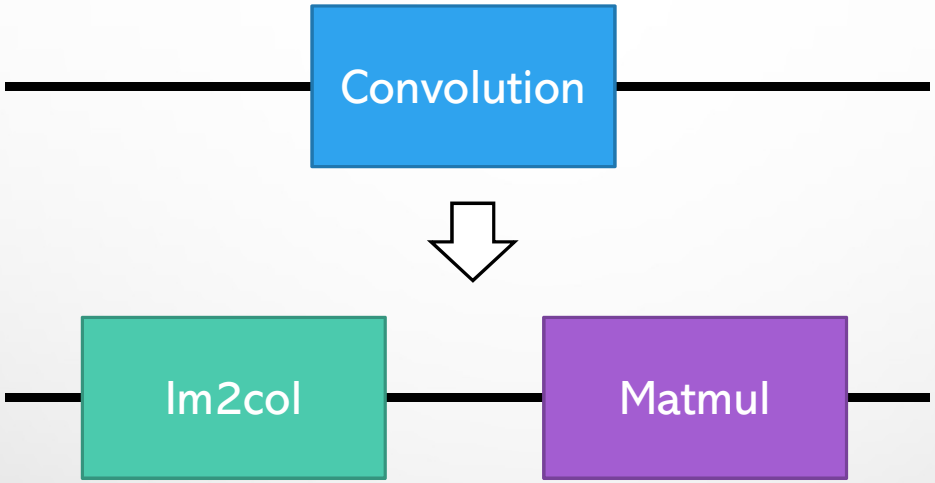
# PARALLELIZING CONV LAYERS

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                       float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
     for(int m = 0;  m < M;  m++)        // for each output feature map
       for(int h = 0; h < H_out; h++)          // for each output element
         for(int w = 0; w < W_out; w++) {
           Y[n, m, h, w] = 0;
           for(int c = 0;  c < C; c++)          // sum over all input feature maps
             for(int p = 0; p < K; p++)         // KxK  filter
               for(int q = 0; q < K; q++)
                 Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
       }
}
```

- Very difficult to parallelize these dimensions

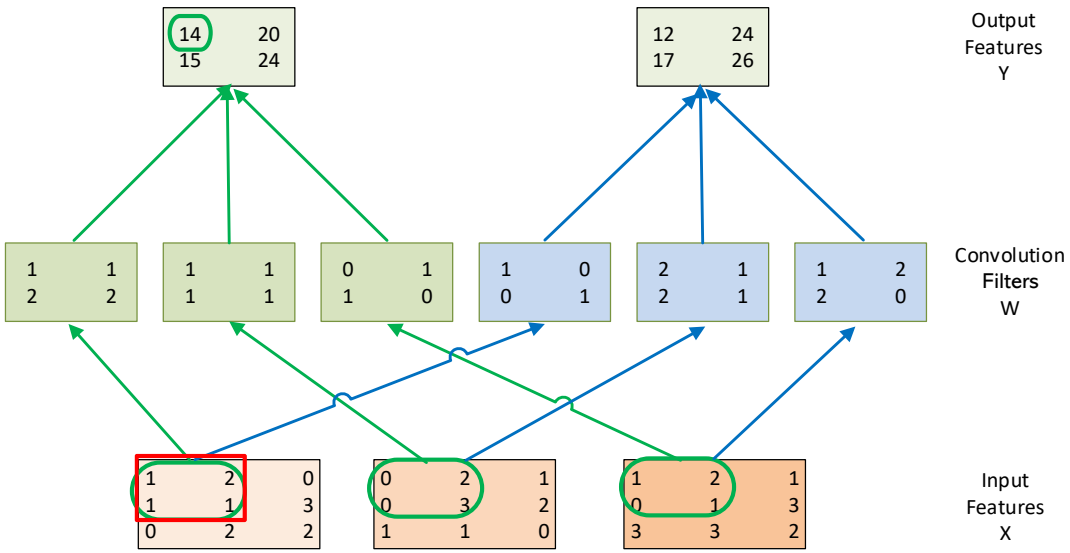
# CONV ~= MATRIX MULTIPLICATION

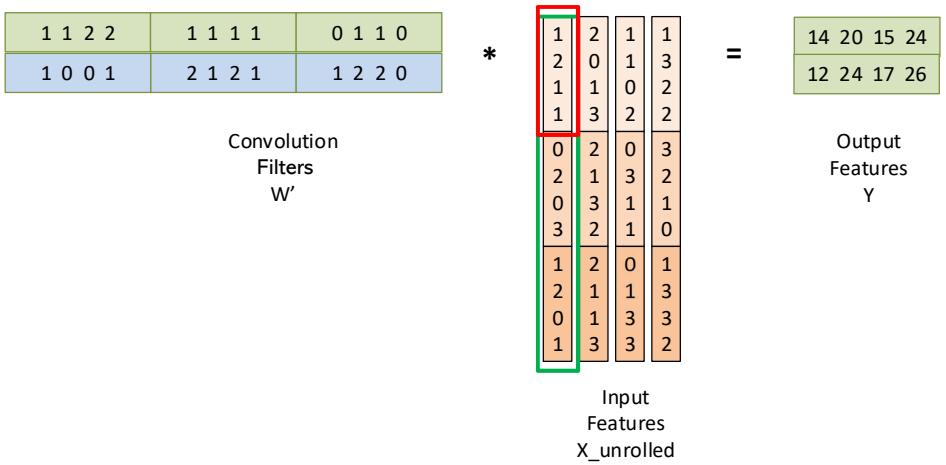
- Im2col (Image to column)



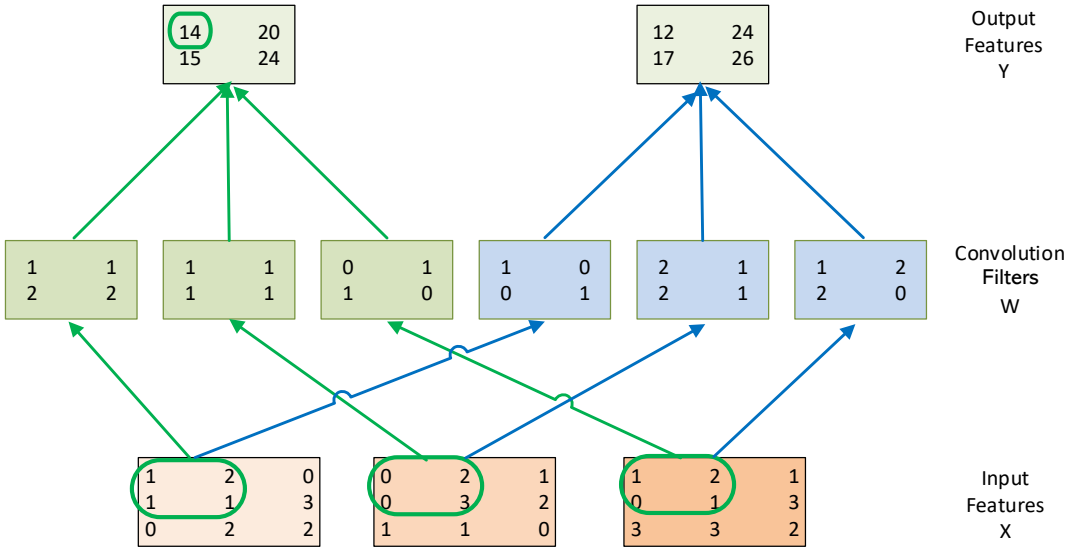

- Partially because matmul is a well-studied application
  - Many libraries, GPUs, and hardware accelerators
- Matrix multiplication is a set of dot products
  - And so is convolution

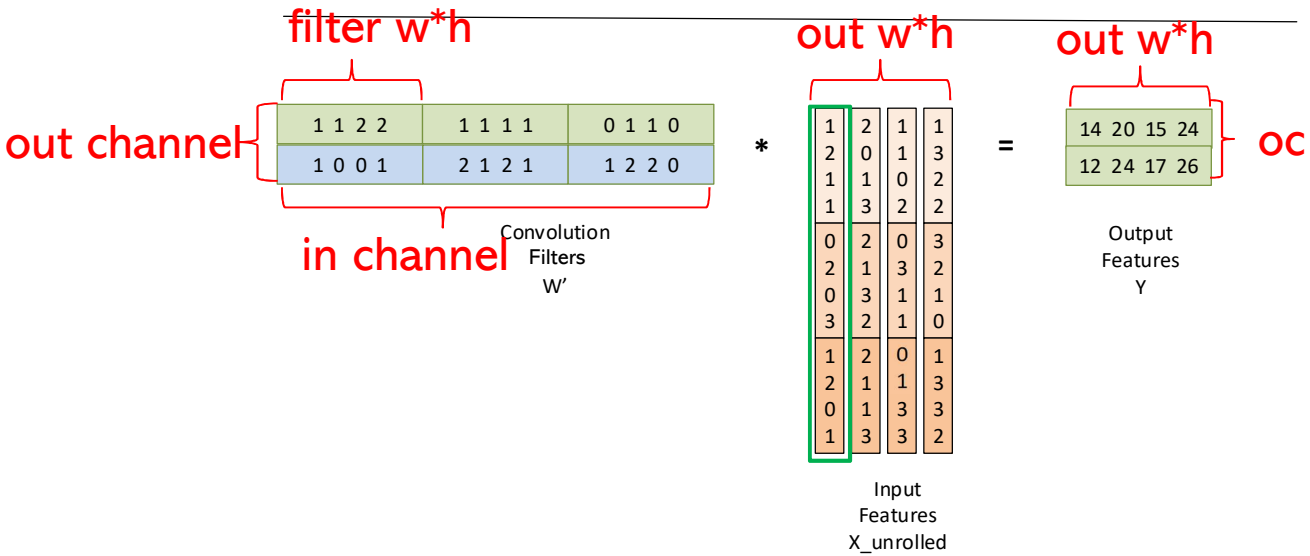
# CONVOLUTION WITH MATMUL (IM2COL)



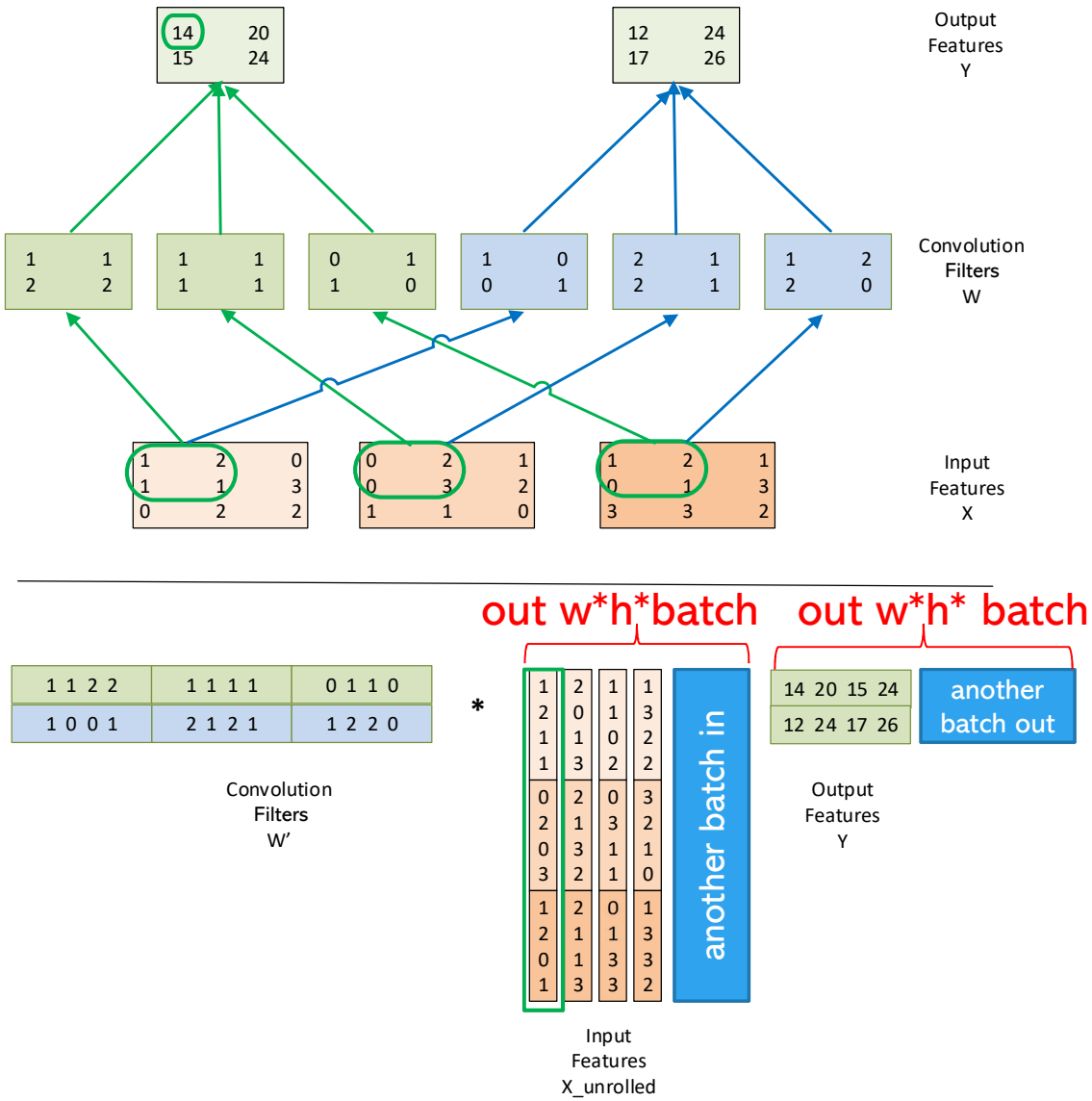



# CONVOLUTION WITH MATMUL (IM2COL)



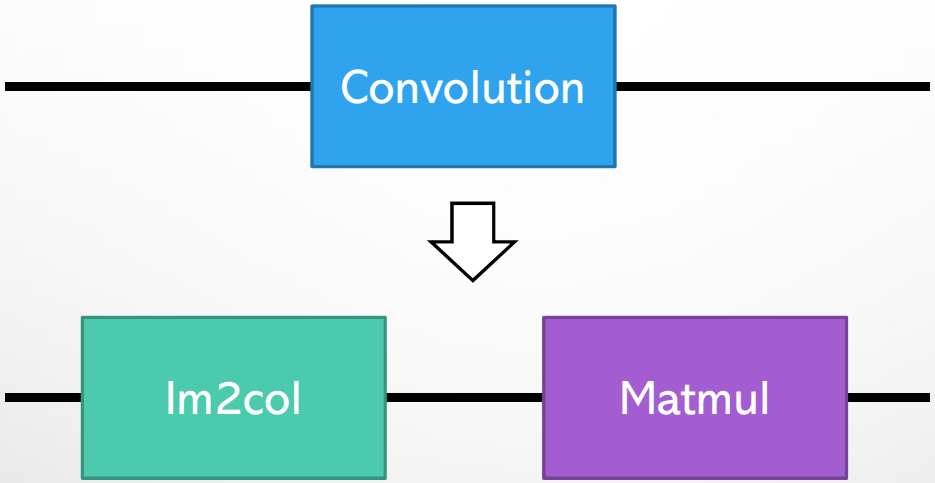



# CONVOLUTION WITH MATMUL (IM2COL)



# PARALLELIZING CONV ~= PARALLELIZING MATRIX MULTIPLICATION

- Im2col (Image to column)



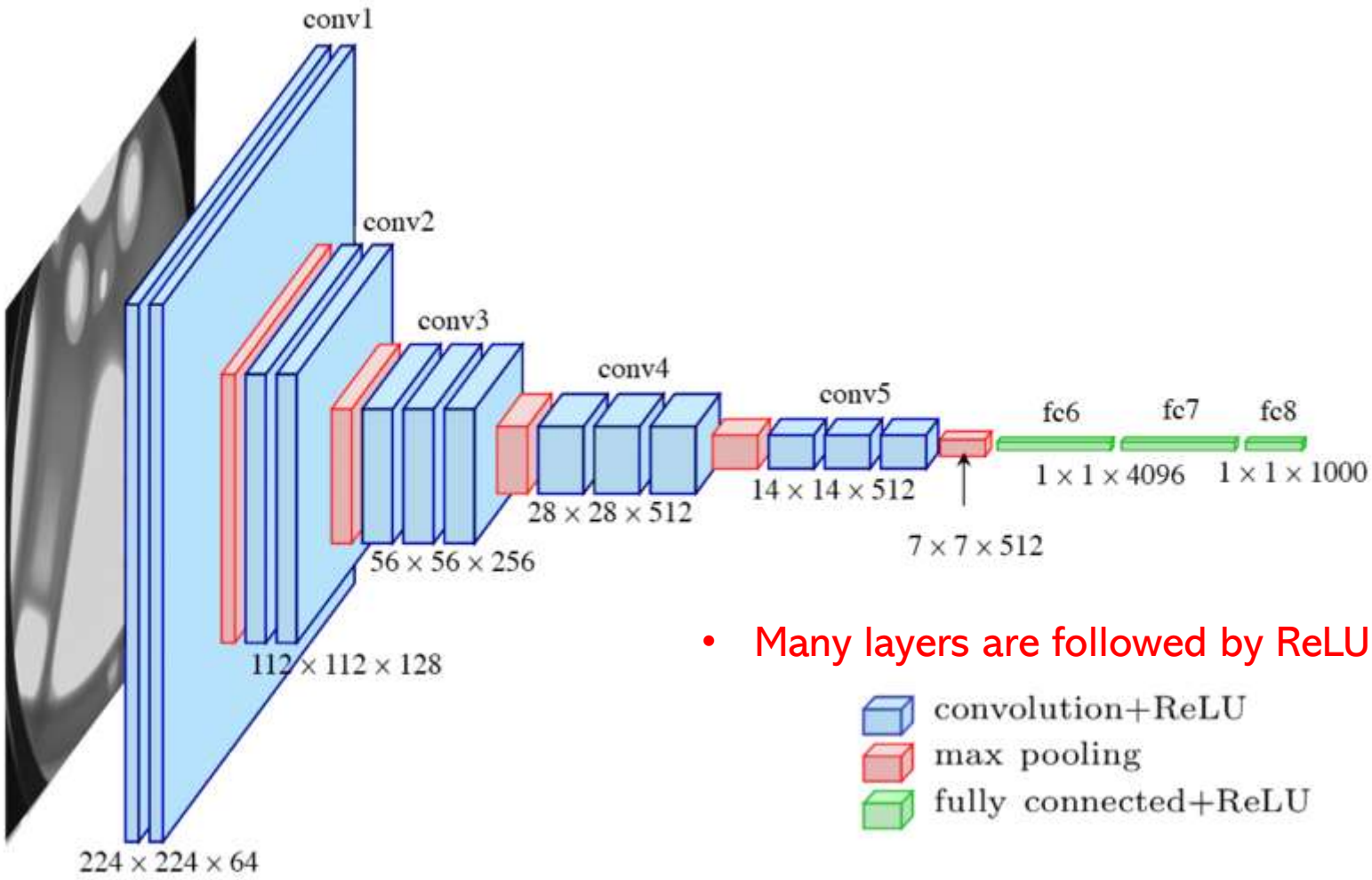

- Im2col makes duplicate entities of input features
- Same amount of computation, but more memory capacity
- Depending on implementation, same or more memory accesses
- But you can reuse your parallelized matrix multiplication
- (!) Carefully parallelizing the conv_forward can be better in some cases
  - Use with caution and thorough analysis

# CONTENT

- DNN components
- Parallelizing DNNs
- An extra (but a common) technique: layer fusion

# FUSION EXAMPLE: RELU

- VGGNet (ImageNet challenge 2014 2nd place)




- Many layers are followed by ReLU



https://clever-keller-127e62.netlify.app/blog/what-is-vggnet/clip_image001.png

# ReLU

```
void ReLU(int N, int M, int C, int H, int W, int K, //KxK filter
                                        float* X, float* W, float* Y)
{
   for(int n = 0;  n < N;  n++)
     for(int m = 0;  m < M;  m++)        // for each output feature map
       for(int h = 0; h < H_out; h++)          // for each output element
         for(int w = 0; w < W_out; w++) {
           Y[n, m, h, w] > 0 ? : Y[n, m, h, w]=0
        }
}
```

- Read everything, simple computation

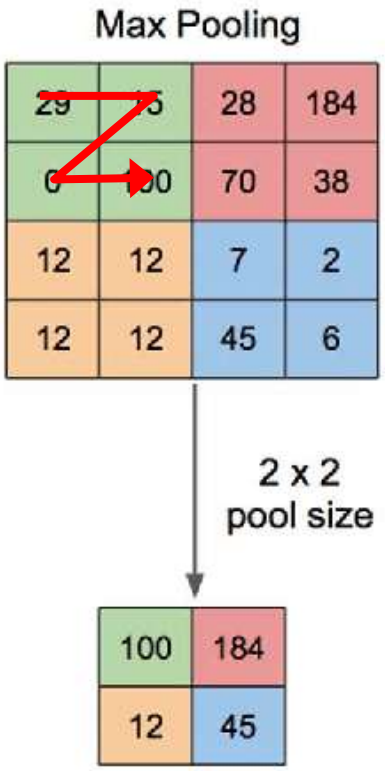
# CONV+RELU

```
void convLayer_forward(int N, int M, int C, int H, int W, int K, //KxK filter
                                                float* X, float* W, float* Y)
{
   int m, c, h, w, p, q;
   int H_out = H – K + 1;
   int W_out = W – K + 1;
   for(int n = 0;  n < N;  n++)
    for(int m = 0;  m < M;  m++)        // for each output feature map
      for(int h = 0; h < H_out; h++)          // for each output element
        for(int w = 0; w < W_out; w++) {
           Y[n, m, h, w] = 0;
           for(int c = 0;  c < C; c++)          // sum over all input feature maps
             for(int p = 0; p < K; p++)         // KxK  filter
                for(int q = 0; q < K; q++)
                    Y[n, m, h, w] += X[n, c, h + p, w + q] * W[m, c, p, q];
           Y[n, m, h, w] > 0 ? : Y[n, m, h, w]=0
        }
}
```



- Fusing two layers save an extra memory access per element

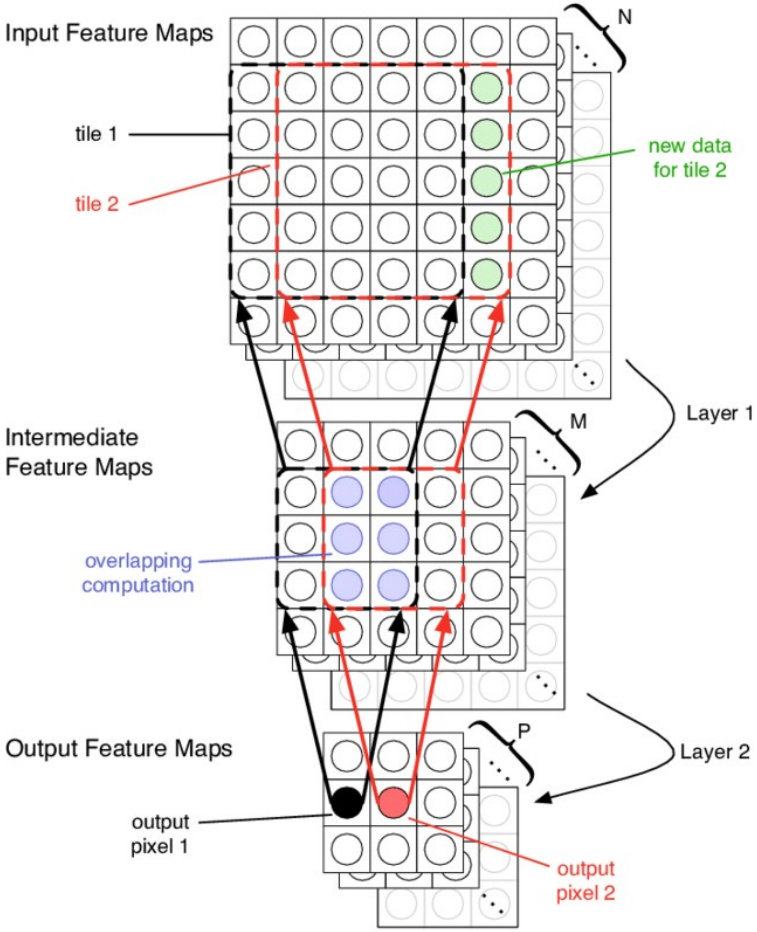
# CONV+POOL




- conv+pool is also possible,
  but requires changing the convolution order
  (or have a large enough cache)

# CONV+CONV

- Fusing multiple conv layers (and many relu, pool layers in between) is also possible, with the right ordering




- Self-study: Fused-layer CNN accelerators, MICRO 2016 [https://ieeexplore.ieee.org/document/7783725](https://ieeexplore.ieee.org/document/7783725)

# CONCLUSION

- To implement a DNN..
  - Implement direct convolution
  - Implement im2col based convolution
  - Implement pooling
  - Implement normalization
  - Implement ReLU
  - Bam! You’re done ☺
  - Optimize whatever you can for the best performance

# GENERATIVE MODELS: LLM AND DIFFUSION

ChatGPT

ChatGPT 4o

Could you make an 1-hour ppt slidedeck for my SNU Astronomy Colloquium tomorrow? I am Jinho Lee at https://aisys.snu.ac.kr/. No more questions and give me a pptx file

I have prepared a 1-hour PowerPoint presentation for your upcoming lecture at the Seoul National University Astronomy Colloquium on March 27, 2025. The presentation is titled "Principles of Generative AI and How to Use, Optimize, and Improve It," aligning with the scheduled topic.

DALL·E

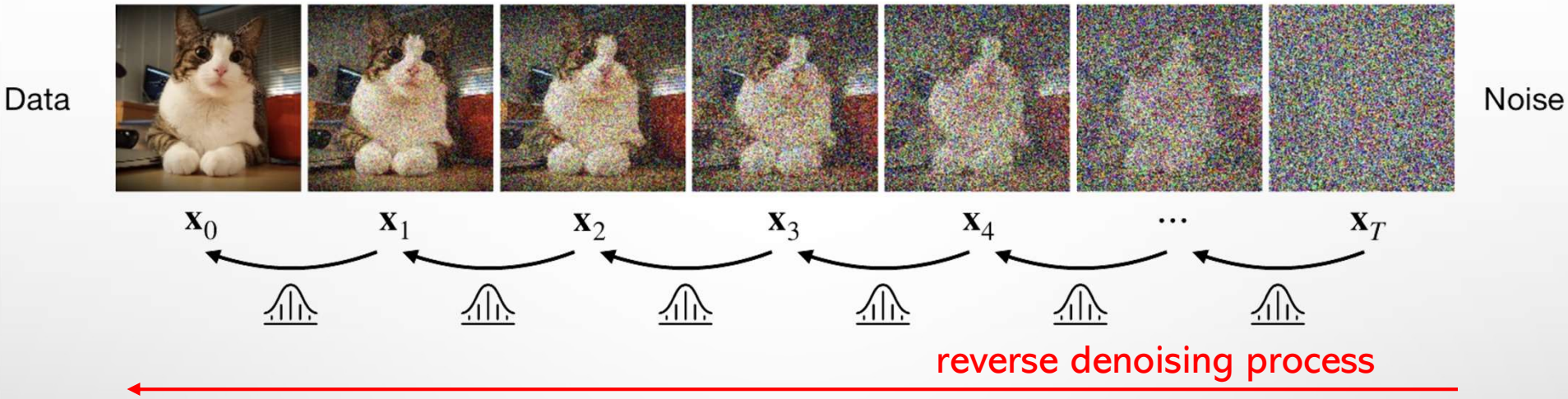
***A close-up portrait of a man with Planet Earth as his head. His entire head is a .....***

# DIFFUSION MODELS

- Start from a clean image
- Keep adding small gaussian noiseses until it's a complete noise

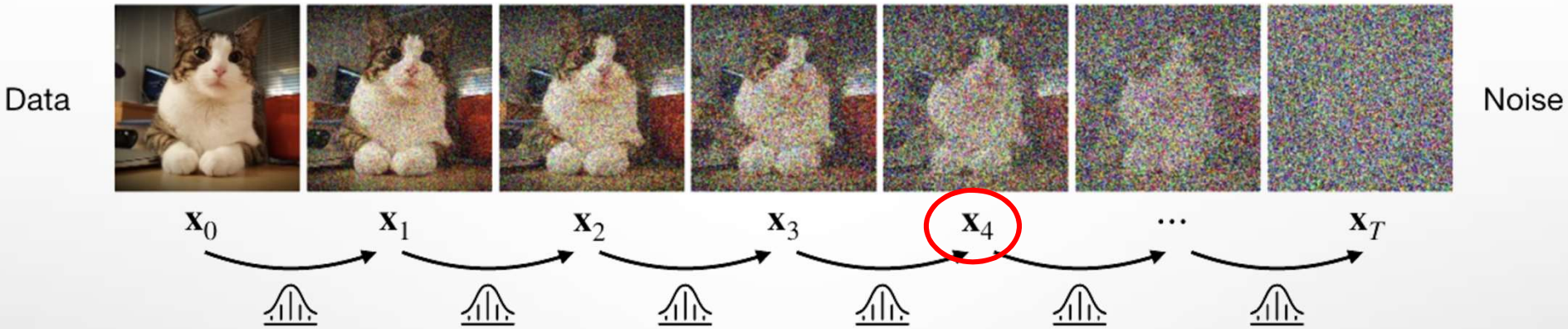
https://cvpr2022-tutorial-diffusion-models.github.io/

# DIFFUSION MODELS




- Let the model learn to denoise
- Somewhat resembles the auto-regressive decoding of LLMs
- Now, give any noise to the model and it will be converted into an image

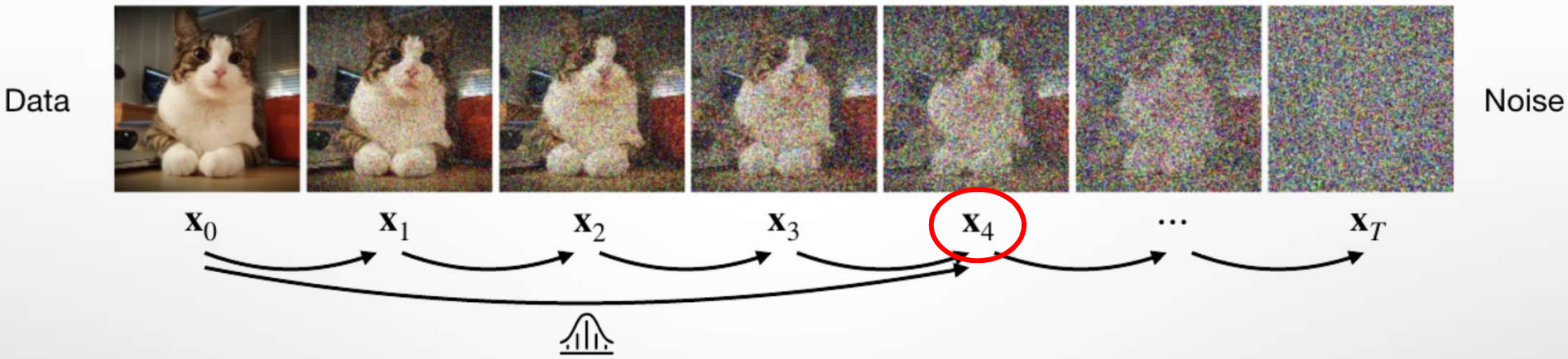
# TRAINING DIFFUSION MODELS




- To add randomness, we pick a random *t* and let the model predict its noise
- $X_0 \rightarrow X_4$ can be done in a single shot because a sum of gaussians is also a gaussian

$$\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I}) + \mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I}) = \mathcal{N}(\mathbf{0}, (\sigma_1 + \sigma_2)^2\mathbf{I})$$

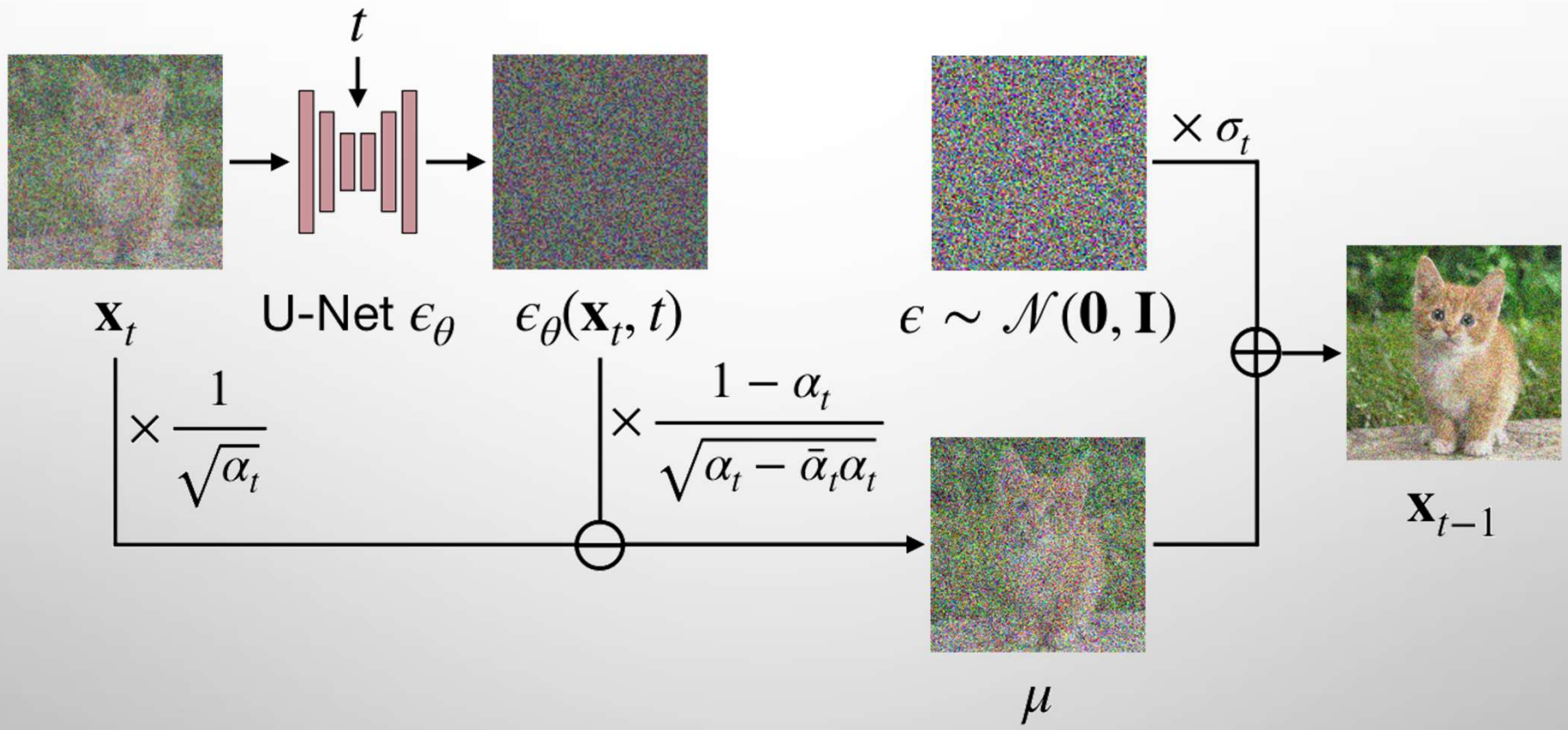
# TRAINING DIFFUSION MODELS




- To add randomness, we pick a random *t* and let the model predict its noise
- $X_0 \rightarrow X_4$ can be done in a single shot because a sum of gaussians is also a gaussian

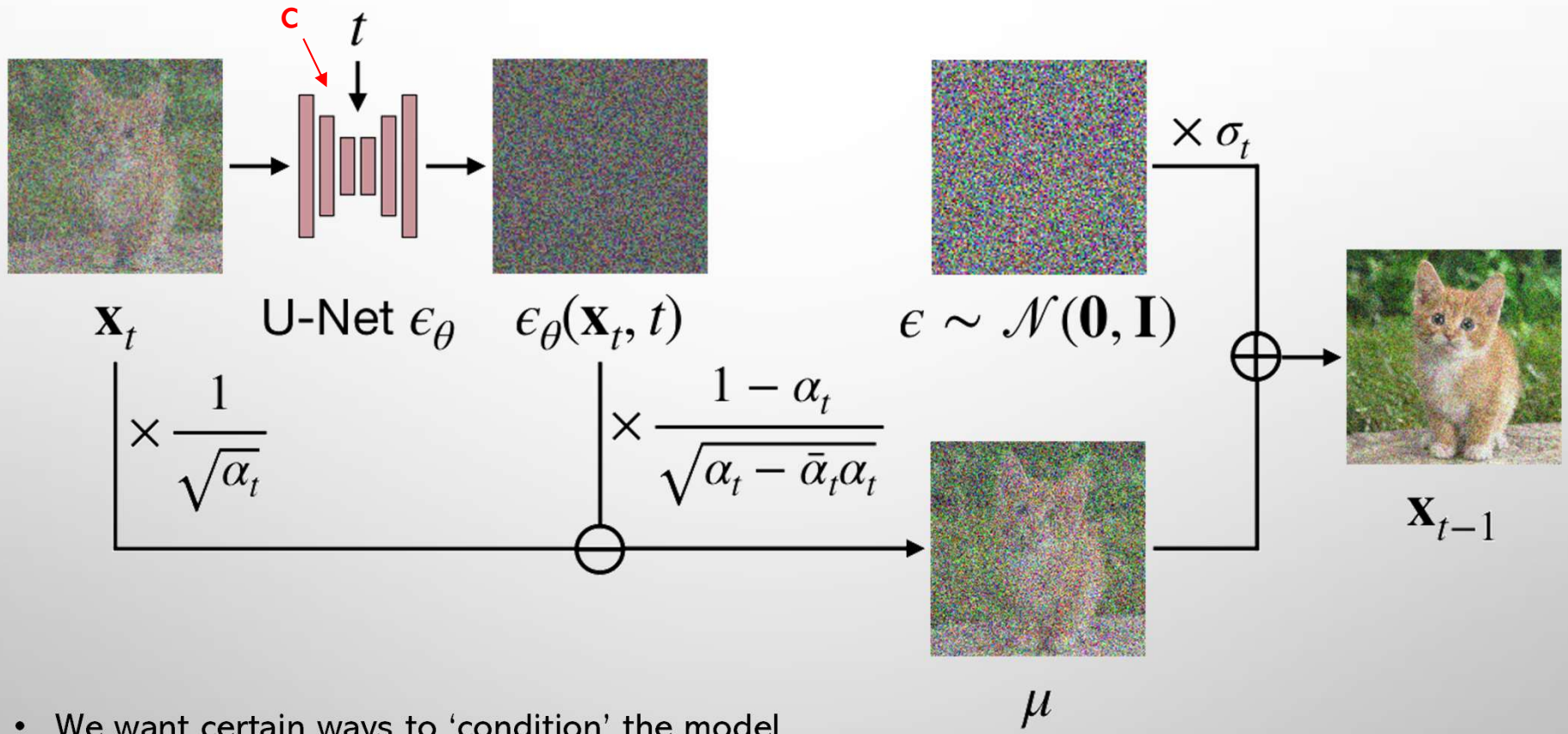
$$\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I}) + \mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I}) = \mathcal{N}(\mathbf{0}, (\sigma_1 + \sigma_2)^2\mathbf{I})$$

# GENERATING (SAMPLING) IMAGE WITH A DIFFUSION MODEL



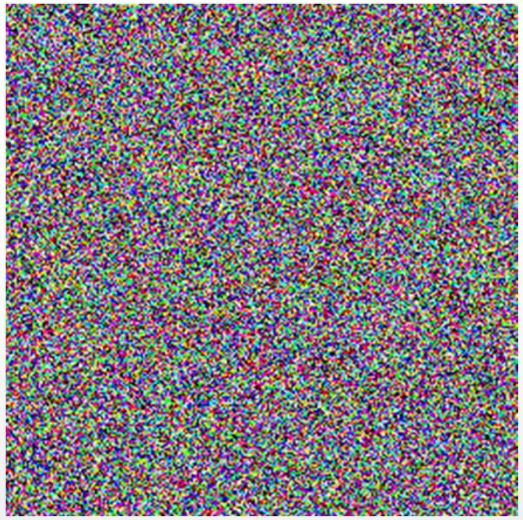

- An NN called U-Net is used (Transformers nowadays)
- Do this for ~1000 steps until we get an image

# EXTENDING TO CONDITIONAL MODELS




- We want certain ways to 'condition' the model
- Classes - cat / dog / airplane / …
- Text – "A photo of myself being a celebrity
- Image – modifying an image

# OVER MULTIPLE ITERATIONS

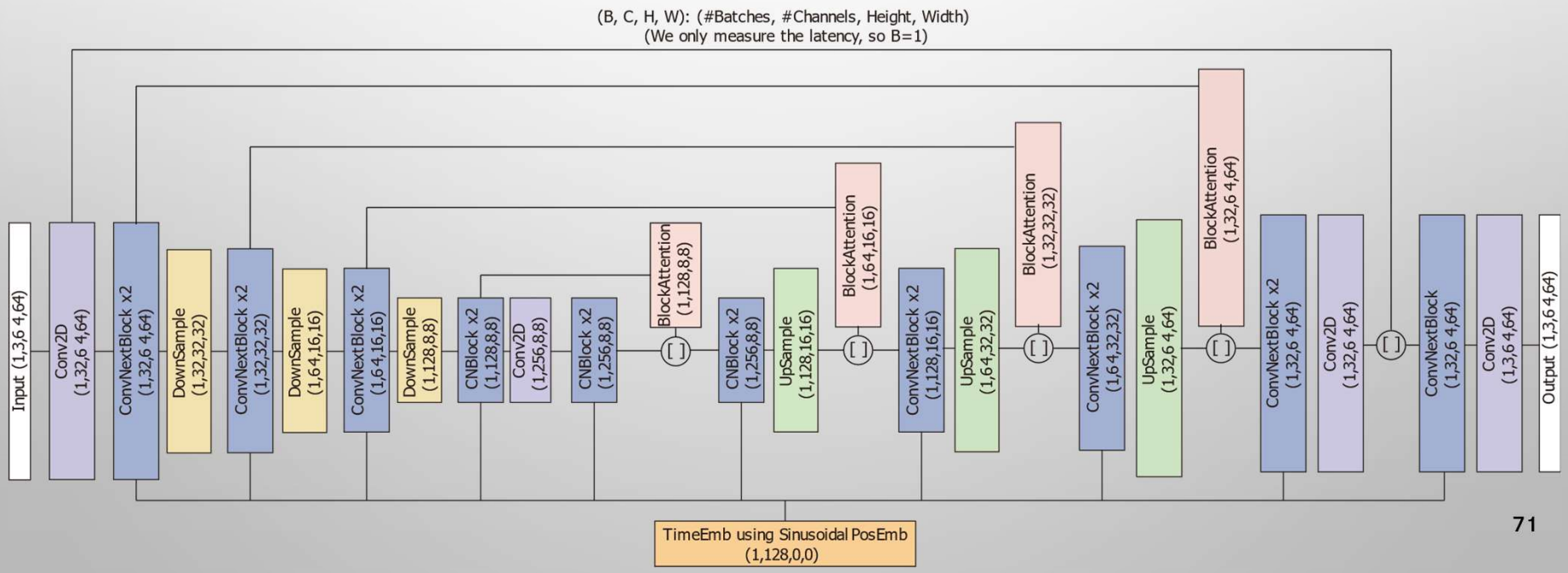


- From a noisy image to cleaner image
- This whole procedure happens during inference (~1000 steps)

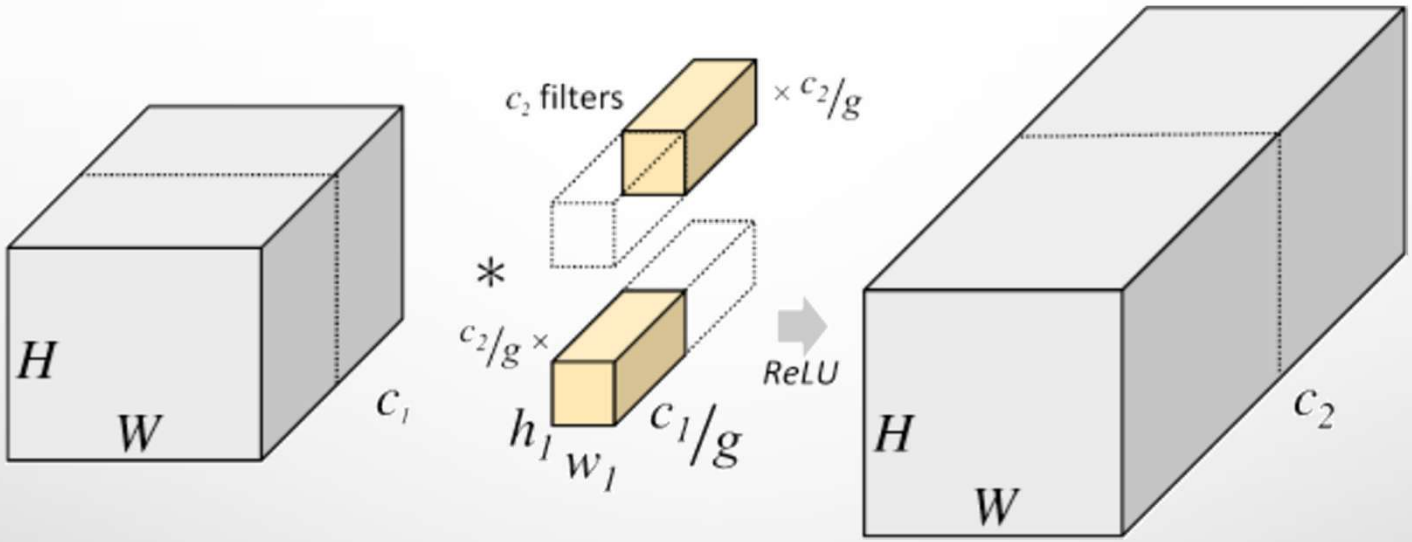
# HW3: DIFFUSION MODEL

A simple, easy dataset

- Convolution
  - w/ stride for downsampling
- Grouped convolution
- Layer normalization
- Attention
  - Not the one used for LLMs, but something we adopted only using conv operations
- Upsampling
- Why/how it works does not matter. Focus only on the computational patterns



# GROUPED CONVOLUTION




- A popular method for reducing the number of computations
- $K^2C^2HW \rightarrow K^2C^2HW/number_of_groups$
- This is used in ConvNext layers
  - Grouped convolution의 설명 및 PyTorch 예시
  - A ConvNet for the 2020s [Liu et al., CVPR 2022]

# REMEMBER, AND BE CAREFUL FOR THE PLAGIARISM

- The moment you look at other's code in any way, that becomes cheating
- (!) You have an obligation to maintain security of your code
  - Do not upload your code in a public repo
  - **When leaving your computer, lock your screen**, so that no one can access it
  - You are guilty if you leave anything unattended